

**PREDIKSI PRESTASI AKADEMIK MAHASISWA BERDASARKAN
STRATEGI BELAJAR DARING MENGGUNAKAN HYBRID MACHINE
LEARNING**

Skripsi

Diajukan Sebagai Salah Satu Syarat Untuk Memperoleh Gelar Sarjana Komputer



Diajukan Oleh:

Abdullah Dzaki Zeky Za'in

1313620010

Prodi Ilmu Komputer

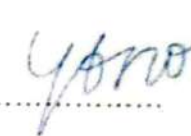
Fakultas Matematika dan Ilmu Pengetahuan Alam

Universitas Negeri Jakarta

2026

LEMBAR PERSETUJUAN HASIL SIDANG SKRIPSI
PREDIKSI PRESTASI AKADEMIK MAHASISWA BERDASARKAN
STRATEGI BELAJAR DARING MENGGUNAKAN HYBRID MACHINE
LEARNING

Nama : Abdullah Dzaki Zeky Za'in
NIM : 1313620010

	Nama	Tanda Tangan	Tanggal
Penanggung Jawab Dekan	: <u>Dr. Hadi Nasbey, S.Pd., M.Si.</u> NIP. 197909162005011004	 	
Wakil Penanggung Jawab Wakil Dekan I	: <u>Dr. Meiliasari, S.Pd., M.Sc.</u> NIP. 197706152003121001		
Ketua Penguji	: <u>Med Irzal, M.Kom.</u> NIP. 197706152003121001		20-01-2026
Sekretaris	: <u>Ersa Resita, S.Tr.T., M.Tr.Kom.</u> NIP. 198811022022031002		21-01-2026
Anggota Pembimbing I	: <u>Dr. Ria Arafiyah, M.Si.</u> NIP. 197511212005012004		21-01-2026
Pembimbing II	: <u>Drs. Mulyono, M.Kom.</u> NIP. 196605171994031003		21-01-2026
Penguji Ahli	: <u>Dr. Vera Maya Santi, S.Si, M.Si.</u> NIP. 197905312005012006		20-01-2026

Dinyatakan lulus ujian skripsi tanggal: 19-01-2026

Lembar Pengesahan

Dengan ini, saya mahasiswa Fakultas Matematika dan Ilmu Pengetahuan Alam,
Universitas Negeri Jakarta.

Judul penelitian : Prediksi Prestasi Akademik Mahasiswa Berdasarkan Strategi
Belajar Daring Menggunakan *Hybrid Machine Learning*

Nama Mahasiswa : Abdullah Dzaki Zeky Za'in

No. Registrasi : 1313620010

Program Studi : Ilmu Komputer

Menyatakan bahwa Skripsi ini siap untuk diajukan sidang skripsi.

Disetujui:

Dosen Pembimbing

Dosen Pembimbing 1



Dr. Ria Arafiah, M. Si

NIP. 197511212005012004

Dosen Pembimbing 2



Drs. Mulyono, M. Kom

NIP. 196605171994031003

Diketahui:

Koordinator Program Studi



Dr. Ria Arafiah, M. Si

NIP. 197511212005012004

LEMBAR PERNYATAAN ORISINALITAS

Yang bertanda tangan dibawah ini:

Nama : Abdullah Dzaki Zeky Za'in

No. Registrasi : 1313620010

Program Studi : Ilmu Komputer

Fakultas : Fakultas Matematika dan Ilmu Pengetahuan Alam

Judul Skripsi:

**PREDIKSI PRESTASI AKADEMIK MAHASISWA BERDASARKAN STRATEGI
BELAJAR DARING MENGGUNAKAN HYBRID MACHINE LEARNING**

Menyatakan bahwa benar skripsi ini Adalah hasil karya saya sendiri. Apabila saya mengutip karya orang lain, maka saya mencantumkan sumbernya dengan ketentuan yang berlaku. Saya bersedia menerima sanksi dari Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Negeri Jakarta apabila terbukti melakukan Tindakan plagiat.

Demikian surat ini saya buat dengan sebenarnya.

Jakarta, 18 Februari 2026



Abdullah Dzaki Zeky Za'in

1313620010

ABSTRAK

Abdullah Dzaki Zeky Za'in-1313620010. Prediksi Prestasi Akademik Mahasiswa Berdasarkan Strategi Belajar Daring Menggunakan *Hybrid Machine Learning*. Program Studi Ilmu Komputer, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Negeri Jakarta, Januari 2026.

Perkembangan pembelajaran daring menuntut mahasiswa memiliki kemampuan regulasi diri yang baik agar mampu mempertahankan prestasi akademik. Penelitian ini bertujuan untuk memprediksi prestasi akademik mahasiswa berdasarkan strategi belajar daring menggunakan pendekatan Hybrid Machine Learning dengan metode stacking ensemble. Model yang dibangun mengombinasikan Random Forest dan Logistic Regression sebagai base learner, serta Logistic Regression sebagai meta-learner untuk menghasilkan prediksi yang lebih stabil dan akurat. Data yang digunakan merupakan data sekunder yang diperoleh dari Mendeley Data, terdiri atas 577 responden dengan 39 fitur yang mencakup variabel demografi dan enam dimensi strategi belajar daring berdasarkan Online Self-Regulated Learning Questionnaire (OSLQ). Tahapan penelitian meliputi data cleaning, encoding, normalisasi, penanganan ketidakseimbangan data menggunakan SMOTE, pembangunan model, serta evaluasi menggunakan metrik akurasi, presisi, recall, dan F1-Score. Hasil penelitian menunjukkan bahwa model hybrid mencapai akurasi tertinggi sebesar 71,96%, lebih baik dibandingkan model tunggal Random Forest sebesar 71,03% dan Logistic Regression sebesar 67,29%. Analisis feature importance menunjukkan bahwa strategi belajar daring seperti goal setting, task strategies, help seeking, dan self-evaluation berperan signifikan terhadap prestasi akademik mahasiswa. Temuan ini menunjukkan bahwa pendekatan hybrid stacking berpotensi meningkatkan performa prediksi sekaligus memberikan pemahaman yang lebih komprehensif terhadap faktor-faktor yang memengaruhi keberhasilan pembelajaran daring.

Kata Kunci: *Hybrid Machine Learning, Stacking Ensemble, Random Forest, Logistic Regression, Online Self-Regulated Learning Questionnaire (OSLQ), Prediksi Prestasi Akademik.*

ABSTRACT

Abdullah Dzaki Zeky Za'in-1313620010. Prediction of Students' Academic Performance Based on Online Learning Strategies Using Hybrid Machine Learning. Computer Science Study Program, Faculty of Mathematics and Natural Sciences, Universitas Negeri Jakarta, January 2026.

The development of online learning requires students to possess strong self-regulated learning abilities in order to maintain academic performance. This study aims to predict students' academic performance based on online learning strategies using a Hybrid Machine Learning approach with a stacking ensemble method. The proposed model combines Random Forest and Logistic Regression as base learners, with Logistic Regression as a meta-learner to produce more stable and accurate predictions. The dataset used in this study is secondary data obtained from Mendeley Data, consisting of 577 respondents and 39 features, including demographic variables and six dimensions of online learning strategies measured using the Online Self-Regulated Learning Questionnaire (OSLQ). The research stages include data cleaning, encoding, normalization, handling class imbalance using SMOTE, model construction, and evaluation using accuracy, precision, recall, and F1-score metrics. The results show that the hybrid model achieved the highest accuracy of 71.96%, outperforming the single Random Forest model (71.03%) and Logistic Regression model (67.29%). Feature importance analysis indicates that online learning strategies such as goal setting, task strategies, help seeking, and self-evaluation play a significant role in students' academic performance. These findings demonstrate that the hybrid stacking approach has the potential to improve prediction performance while providing a more comprehensive understanding of factors influencing success in online learning.

Keywords: *Hybrid Machine Learning, Stacking Ensemble, Random Forest, Logistic Regression, Online Self-Regulated Learning (OSLQ), Academic Performance Prediction.*

Kata Pengantar

Puji dan syukur saya panjatkan ke hadirat Allah Swt. karena dengan rahmat dan karunia-Nya penulis dapat menyelesaikan proposal skripsi yang berjudul “Prediksi Prestasi Akademik Mahasiswa Berdasarkan Strategi Belajar Daring Menggunakan *Hybrid Machine Learning*”. Proposal ini dibuat demi memenuhi persyaratan untuk mendapatkan gelar sarjana S1 pada program studi Ilmu Komputer Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Negeri Jakarta.

Mengingat keterbatasan pengetahuan dan pengalaman penulis, dalam penyusunan proposal skripsi ini penulis banyak menerima bantuan, petunjuk, saran, maupun arahan dari berbagai pihak. Oleh karena itu, dengan penuh kerendahan hati dan rasa hormat penulis ucapkan terima kasih yang sebesar-besarnya kepada:

1. Ibu Dr. Ria Arafiah, M.Si. selaku ketua Program Studi Ilmu Komputer sekaligus Dosen Pembimbing Pertama yang telah memberi arahan, dorongan, serta semangat dalam pembuatan proposal skripsi ini.
2. Bapak Drs. Mulyono, M.Kom. selaku Dosen Pembimbing Kedua yang telah memberi arahan, dorongan, dan semangat dalam pembuatan proposal skripsi ini.
3. Orang tua dan keluarga terkasih yang terus-menerus memberikan do’a dan dukungan kepada penulis.
4. Teman-teman yang memberi bantuan dan semangat bagi penulis.

Penulis menyadari sepenuhnya bahwa masih terdapat kekurangan baik dari penyusunan bahasa, pemahaman, dan berbagai aspek lainnya. Oleh karena itu, kritik dan saran akan penulis terima dengan hati terbuka. Penulis berharap proposal skripsi ini dapat bermanfaat bagi pihak yang membutuhkan.

Jakarta, 15 Januari 2026



Abdullah Dzaki Zeky Za'in

DAFTAR ISI

Lembar Pengesahan	ii
ABSTRAK	iii
ABSTRACT	iv
DAFTAR ISI	vi
DAFTAR GAMBAR	ix
DAFTAR TABEL	x
BAB I PENDAHULUAN	1
1.1. Latar Belakang	1
1.2. Rumusan Masalah	3
1.3. Tujuan Penelitian	3
1.4. Batasan Masalah	4
1.5. Manfaat Penelitian	4
BAB II KAJIAN PUSTAKA	6
2.1. Prestasi Akademik	6
2.2. Pembelajaran Daring	7
2.3. <i>Self-Regulated Learning</i>	7
2.4. <i>Exploratory Data Analysis</i>	8
2.4.1. Statistik Deskriptif	9
2.4.2. <i>Univariate Analysis</i>	10
2.4.3. <i>Multivariate Analysis</i>	10
2.5. <i>Data Pre-Processing</i>	11
2.5.1. Deteksi dan Penanganan <i>Outlier</i>	12
2.5.2. <i>Data Encoding</i>	12
2.5.3. <i>Feature Scaling</i>	13
2.5.4. <i>Balancing Data</i>	14
2.6. <i>Synthetic Minority Oversampling Method</i>	14
2.7. <i>Machine Learning</i>	15
2.7.1. <i>Supervised Learning</i>	16
2.7.2. <i>Unsupervised Learning</i>	17

2.7.3.	<i>Reinforcement Learning</i>	18
2.8.	<i>Decision Tree</i>	18
2.9.	<i>Random Forest</i>	23
2.10.	<i>Logistic Regression</i>	25
2.10.1.	Regresi Linear Sederhana.....	26
2.10.2.	Regresi Linear Berganda	26
2.10.3.	<i>Logistic Regression</i>	27
2.11.	<i>Hybrid Machine Learning</i>	29
2.12.	Evaluasi <i>Machine Learning</i>	32
2.13.	Validasi Model <i>Machine Learning</i>	33
BAB III METODE PENELITIAN		35
3.1.	Alat Penelitian	35
3.2.	Tahapan Penelitian	35
3.3.	Pengumpulan dan Penyusunan <i>Dataset</i>	36
3.4.	Tahapan <i>Pre-Processing Data</i>	44
3.5.	Tahapan Pemodelan <i>Hybrid Machine Learning</i>	45
BAB IV		47
HASIL DAN PEMBAHASAN		47
4.1.	<i>Data Exploration</i>	47
4.2.	Data Pre-processing	48
4.3.	Modeling.....	52
4.3.1.	Pembangunan Base Model 1: <i>Random Forest</i>	52
4.3.2.	Pembangunan Base Model 2: <i>Logistic Regression</i>	53
4.3.3.	Pembangunan <i>Meta Learner</i>	54
4.4.	Hasil Penelitian	54
4.5.	Analisis Hasil.....	64
4.6.	Pembahasan	66
BAB V		70
KESIMPULAN DAN SARAN		70
5.1.	Kesimpulan.....	70
5.2.	Saran	71
DAFTAR PUSTAKA		72
LAMPIRAN.....		76

DAFTAR GAMBAR

Gambar 2. 1 Decision Tree	19
Gambar 2. 2. Random Forest	25
Gambar 3. 1. Alur Penelitian Hybrid Machine Learning.....	36
Gambar 4. 1. Diagram Jumlah Mahasiswa per Fakultas.....	48
Gambar 4. 2. Kolom Ordinal dan Nominal.....	50
Gambar 4. 3. Proporsi Kelas Data Latih dan Uji	51
Gambar 4. 4. Confusion Matrix Random Forest.....	56
Gambar 4. 5. Feature Importance Random Forest	57
Gambar 4. 6. Confusion Matrix <i>Logistic Regression</i>	59
Gambar 4. 7. Feature Importance <i>Logistic Regression</i>	60
Gambar 4. 8. Confusion Matrix Meta Learner.....	62
Gambar 4. 9. Feature Importance Meta Learner	63

DAFTAR TABEL

Tabel 2. 1. Elemen Decision Tree	19
Tabel 3. 1. Demografi Mahasiswa	37
Tabel 3. 2. Kuesioner Regulasi Diri dalam Belajar	40
Tabel 4. 1. Kelas IPK	47
Tabel 4. 2. Parameter Random Forest.....	53
Tabel 4. 3. Parameter <i>Logistic Regression</i>	53
Tabel 4. 4. Classification Report Random Forest	55
Tabel 4. 5. Classification Report <i>Logistic Regression</i>	58
Tabel 4. 6. Classification Report Meta Learner	61

BAB I

PENDAHULUAN

1.1. Latar Belakang

Pendidikan tinggi berperan penting dalam mengembangkan potensi mahasiswa agar mampu bersaing di dunia kerja dan berkontribusi bagi masyarakat. Salah satu indikator keberhasilan pendidikan tinggi adalah prestasi akademik mahasiswa, yang umumnya diukur melalui Indeks Prestasi Kumulatif (IPK). IPK mencerminkan kemampuan mahasiswa dalam memahami dan menerapkan pengetahuan yang diperoleh selama studi serta menjadi dasar bagi institusi pendidikan untuk menilai keberhasilan proses pembelajaran (Liu et al., 2025). Prestasi akademik yang baik tidak hanya memberikan peluang karier yang lebih luas bagi mahasiswa, tetapi juga mencerminkan kemampuan mahasiswa dalam mengelola waktu, motivasi, dan strategi belajar secara efektif.

Perkembangan teknologi informasi telah mendorong transformasi signifikan dalam sistem pendidikan, khususnya melalui penerapan pembelajaran daring (*online learning*). Model pembelajaran daring menawarkan fleksibilitas waktu dan tempat, memungkinkan mahasiswa untuk belajar secara mandiri tanpa batasan geografis. Namun, keberhasilan pembelajaran daring sangat bergantung pada kemampuan mahasiswa dalam mengatur diri sendiri dalam pembelajaran (*self-regulated learning*), seperti menetapkan tujuan belajar, mengelola waktu, memantau kemajuan, dan mempertahankan motivasi belajar. Ketidakmampuan mengelola aspek-aspek tersebut sering kali berdampak pada penurunan prestasi akademik (UNESCO, 2022). Oleh karena itu, pemahaman terhadap strategi belajar daring yang efektif menjadi hal krusial untuk mendukung keberhasilan akademik mahasiswa di era digital.

Pandemi COVID-19 memaksa perubahan sistem pendidikan dari tatap muka ke daring. Meskipun pendekatan ini memastikan keberlangsungan proses belajar, banyak institusi pendidikan menghadapi tantangan seperti kesenjangan akses teknologi, keterbatasan interaksi sosial, dan kesulitan mempertahankan fokus

belajar. Dampak ini menimbulkan variasi hasil akademik antar mahasiswa, menunjukkan bahwa keberhasilan belajar daring tidak hanya bergantung pada fasilitas, tetapi juga pada strategi belajar individu yang digunakan. Dengan demikian, penting untuk meneliti hubungan antara strategi belajar daring dan prestasi akademik mahasiswa sebagai dasar dalam perancangan sistem pembelajaran daring yang adaptif dan efektif.

Untuk menganalisis hubungan kompleks antara strategi belajar daring dan prestasi akademik mahasiswa, pendekatan tradisional seperti analisis statistik sering kali tidak memadai karena keterbatasan dalam menangkap pola non-linear dari data multidimensional. Dalam konteks ini, *machine learning* (ML) menawarkan solusi yang lebih kuat karena mampu mengenali pola tersembunyi dalam data dan melakukan prediksi berbasis pembelajaran dari data historis. Berbagai penelitian telah memanfaatkan algoritma seperti *Decision Tree*, *Random Forest*, *Support Vector Machine*, dan *Logistic Regression* untuk memprediksi capaian akademik mahasiswa. Namun, sebagian besar penelitian tersebut hanya menggunakan model tunggal yang terkadang menghasilkan akurasi terbatas dan sulit untuk diinterpretasikan.

Pendekatan *hybrid machine learning* hadir sebagai alternatif yang dapat mengkombinasikan beberapa algoritma untuk memanfaatkan keunggulan masing-masing model, sehingga meningkatkan akurasi dan stabilitas hasil prediksi (Ozdemir et al., 2025). Salah satu kombinasi yang berpotensi memproses data dengan baik adalah penggabungan *Random Forest* dan *Logistic Regression* untuk memprediksi kelas IPK mahasiswa. *Random Forest* memiliki keunggulan dalam melakukan perhitungan *feature importance*, yang mampu mengidentifikasi variabel paling berpengaruh terhadap hasil prediksi (Saarela et al., 2021). Sementara itu, *Logistic Regression* mampu menghasilkan model yang lebih mudah diinterpretasikan dan efisien dalam menjelaskan hubungan antar variabel prediktor dan variabel target (Molnar, 2020). Integrasi kedua model ini memungkinkan peneliti memperoleh hasil prediksi yang akurat sekaligus memahami faktor-faktor strategi belajar yang paling berpengaruh terhadap prestasi akademik mahasiswa.

Berdasarkan latar belakang tersebut, penelitian ini berfokus pada pengembangan model *hybrid machine learning* dua tahap untuk memprediksi prestasi akademik mahasiswa berdasarkan strategi belajar daring yang diukur melalui *Online Self-Regulated Learning Questionnaire* (OSLQ). Tahap pertama *hybrid machine learning* menggunakan *Random Forest*, sedangkan tahap kedua menggunakan *Logistic Regression* sebagai model prediktif utama dan *meta-learner* dalam arsitektur *stacking*. Pendekatan ini diharapkan mampu menghasilkan model yang tidak hanya akurat, tetapi juga dapat memberikan wawasan tentang strategi belajar daring yang paling berkontribusi terhadap keberhasilan akademik mahasiswa di era digital.

1.2. Rumusan Masalah

Berdasarkan permasalahan yang telah dideskripsikan pada latar belakang, dapat diidentifikasi beberapa masalah yang dirumuskan untuk membantu peneliti dalam mengarahkan penelitian ini, yaitu:

1. Strategi belajar daring apa yang paling berpengaruh pada prestasi akademik mahasiswa?
2. Bagaimana model *hybrid machine learning* dapat digunakan untuk memprediksi prestasi akademik mahasiswa berdasarkan strategi belajar daring?
3. Bagaimana performa model yang dibangun dibandingkan model tunggal?

1.3. Tujuan Penelitian

Berdasarkan latar belakang yang telah dideskripsikan, dapat disimpulkan fokus atau arah penelitian yang menjadi tujuan akhir dalam penelitian ini, yaitu:

1. Mengidentifikasi dimensi strategi belajar daring yang paling berpengaruh pada prestasi akademik mahasiswa.
2. Membangun model *hybrid machine learning* dengan metode *stacking* antara model *Random Forest* dan *Logistic Regression* sebagai *base model* serta *Logistic Regression* sebagai *meta-learner* untuk memprediksi prestasi akademik mahasiswa berdasarkan strategi belajar daring.
3. Mengevaluasi model *hybrid machine learning* dibangun dalam memprediksi prestasi akademik mahasiswa.

1.4. Batasan Masalah

Demi menjaga agar penelitian ini tidak keluar dari pokok permasalahan, maka ditetapkan Batasan masalah sebagai berikut:

1. Data yang digunakan dalam penelitian ini adalah yang dikumpulkan oleh Bapak Dr. Irwanto, M.Pd., yang berasal dari mahasiswa di beberapa fakultas Universitas Jakarta yang mencakup data variabel demografi dan jawaban kuesioner OSLQ.
2. Prestasi akademik mahasiswa yang menjadi variabel target dalam penelitian ini didefinisikan berdasarkan nilai Indeks Prestasi Kumulatif (IPK) yang tersedia dalam *dataset*. Penelitian ini tidak mempertimbangkan aspek lain dari prestasi akademik yang tidak tercakup dalam *dataset* tersebut.
3. Pembangunan model untuk *base learner* dalam arsitektur *hybrid machine learning* akan memanfaatkan implementasi algoritma yang tersedia dalam pustaka (*library*) *machine learning* standar.
4. Analisis strategi belajar daring akan berfokus pada dimensi-dimensi yang tercakup pada kuesioner *Online Self-Regulated Learning Questionnaire* (OSLQ).
5. Model prediksi yang dihasilkan dalam penelitian ini divalidasi berdasarkan *dataset* yang digunakan. Generalisasi model untuk populasi mahasiswa di luar karakteristik *dataset* yang ada mungkin memerlukan studi lebih lanjut dengan data yang lebih bervariasi.

1.5. Manfaat Penelitian

Dengan dilaksanakannya penelitian ini, diharapkan dapat memberi manfaat sebagai berikut:

1. Bagi program studi Ilmu Komputer UNJ, diharapkan model klasifikasi ini dapat memberi wawasan yang bermanfaat sehingga program studi dapat mengembangkan sistem pembelajaran yang lebih adaptif.
2. Bagi mahasiswa program studi Ilmu Komputer UNJ, diharapkan penelitian ini dapat menjadi gambaran dalam menentukan strategi belajar yang efektif dan efisien.

3. Bagi peneliti, agar dapat menyelesaikan pendidikan tingkat sarjana di Universitas Negeri Jakarta, dengan menyelesaikan penelitian sebagai bentuk implementasi dari ilmu yang telah dipelajari semasa kuliah dan diharapkan dapat bermanfaat bagi pihak yang terkait dalam penelitian ini.

BAB II

KAJIAN PUSTAKA

2.1. Prestasi Akademik

Prestasi akademik merupakan indikator keberhasilan mahasiswa dalam proses pembelajaran yang direpresentasikan melalui capaian akademik formal selama masa studi. Prestasi akademik sering diukur menggunakan Indeks Prestasi Kumulatif (IPK) sebagai ukuran agregat yang mencerminkan pencapaian mahasiswa pada berbagai mata kuliah. Nilai IPK atau *Cumulative Grade Point* (GPA) digunakan sebagai representasi capaian akademik mahasiswa pada penelitian yang berjudul “*Data on Undergraduate Students’ Self Regulation in Online and Blended Learning Environments During the COVID-19 Pandemic in Indonesia*” (Irwanto, 2024). Pada *dataset* tersebut, IPK disajikan dalam bentuk kategori tingkat capaian akademik, sehingga dalam penelitian ini, prestasi akademik diperlakukan sebagai variabel kategorikal dan digunakan sebagai variabel target dalam pemodelan klasifikasi.

Prestasi akademik mahasiswa dalam lingkungan pembelajaran daring berkaitan erat dengan kemampuan *Self-Regulated Learning*, di mana mahasiswa yang mampu mengatur tujuan, strategi belajar, dan evaluasi diri secara efektif cenderung menunjukkan performa akademik yang lebih baik (Bernard et al., 2009). Penggunaan IPK sebagai variabel target dalam penelitian ini relevan untuk mengkaji kontribusi strategi belajar daring yang diukur melalui instrumen OSLQ. Oleh karena itu, pendekatan *hybrid machine learning* dapat digunakan dalam penelitian ini untuk mengombinasikan keunggulan beberapa algoritma dalam memodelkan hubungan linear maupun non-linear, sehingga diharapkan mampu meningkatkan akurasi prediksi prestasi akademik mahasiswa serta memberi gambaran mengenai faktor-faktor yang berkontribusi terhadap keberhasilan pembelajaran daring.

2.2. Pembelajaran Daring

Perkembangan teknologi informasi telah membawa perubahan besar dalam dunia pendidikan, terutama dalam penerapan pembelajaran berbasis daring. Pembelajaran daring didefinisikan sebagai proses pembelajaran yang berlangsung dengan memanfaatkan jaringan internet, di mana interaksi antara pengajar dan peserta didik dilakukan melalui berbagai platform digital tanpa kehadiran fisik di ruang kelas (Moore et al., 2011). Model pembelajaran ini memungkinkan fleksibilitas waktu dan tempat, sehingga mahasiswa dapat mengakses materi belajar kapan saja dan di mana saja. Pembelajaran daring menawarkan kemudahan akses, efisiensi waktu, serta peluang belajar mandiri yang lebih tinggi dibandingkan dengan pembelajaran tatap muka konvensional, di sisi lain, pembelajaran daring juga menimbulkan tantangan seperti menurunnya motivasi belajar, rendahnya keterlibatan mahasiswa, dan meningkatnya risiko prokrastinasi (Song et al., 2004).

Keberhasilan pembelajaran daring sangat bergantung pada kemampuan mahasiswa untuk mengelola proses belajarnya sendiri. Pembelajaran daring memiliki karakteristik otonomi yang tinggi, maka kemampuan regulasi diri menjadi faktor yang sangat menentukan keberhasilan mahasiswa dalam proses belajar daring (Barnard et al., 2009). Kemampuan regulasi diri seperti kemampuan menetapkan tujuan belajar, mengatur waktu, monitor kemajuan, serta mengevaluasi hasil belajar, kemampuan ini dikenal dengan istilah *Self-Regulated Learning* (SRL). Oleh karena itu, SRL menjadi faktor penting yang menentukan keberhasilan akademik mahasiswa, karena membantu mereka tetap termotivasi dan terarah meski dalam lingkungan belajar yang fleksibel dan kurangnya kontrol eksternal.

2.3. *Self-Regulated Learning*

Self-Regulated Learning (SRL) atau pembelajaran yang diatur sendiri merupakan konsep yang menjelaskan kemampuan individu dalam mengelola dan mengarahkan proses belajarnya secara mandiri, SRL melibatkan proses aktif di mana pelajar berpartisipasi secara metakognitif, motivasional, dan perilaku dalam pembelajaran untuk mencapai tujuan akademik (Zimmerman, 2002). Mahasiswa

yang memiliki kemampuan SRL yang baik mampu merencanakan, memantau, dan mengevaluasi proses belajarnya dengan lebih efektif. Zimmerman menjelaskan bahwa SRL terdiri atas tiga fase utama, yaitu *forethought* berupa perencanaan dan penetapan tujuan, *performance* berupa pelaksanaan dan pemantauan strategi belajar, serta *self-reflection* berupa evaluasi dan strategi belajar. pelajar yang efektif tidak hanya mengatur proses kognitif, tetapi juga memelihara motivasi dan mengelola konteks belajarnya (Pintrich, 2000).

Sebagai pengembangan lebih lanjut dari teori SRL, Barnard et al, (2009) mengadaptasikan konsep ini ke dalam lingkungan pembelajaran daring dan campuran. Instrumen *Online Self-Regulated Learning Questionnaire* (OSLQ) dikembangkan untuk mengukur tingkat kemampuan regulasi diri mahasiswa dalam pembelajaran berbasis teknologi. Instrumen ini terdiri dari enam dimensi utama, yaitu *Goal Setting*, *Environment Structuring*, *Task Strategies*, *Time Management*, *Help Seeking*, dan *Self-Evaluation*. Keenam dimensi tersebut menggambarkan bagaimana mahasiswa mengelola pembelajaran daring secara aktif dan reflektif, mahasiswa dengan tingkat SRL yang tinggi akan lebih mampu menyesuaikan strategi belajar terhadap perubahan situasi belajar daring yang menuntut disiplin dan kemandirian tinggi (Barnard et al., 2009)

2.4. Exploratory Data Analysis

Exploratory Data Analysis atau disingkat EDA adalah salah satu bagian dari proses *data science* sebagai strategi dalam menganalisis sebuah data. EDA pertama kali diperkenalkan oleh salah satu ahli statistik yang bernama John W Tukey pada tahun 1961. EDA didefinisikan sebagai pekerjaan detektif, hal ini dikarenakan peneliti mengeksplorasi data tanpa memiliki ide atau bias tentang informasi apa yang akan diperoleh dari data yang akan dianalisis (Vegari & Budi, 2020).

Komponen-komponen dalam EDA meliputi berbagai perhitungan statistik dasar, visualisasi data, hipotesis awal, pemeriksaan asumsi, hingga *reporting*. EDA tidak hanya terpaku pada suatu teknik tertentu saja melainkan eksplorasi data dengan berbagai macam teknik dan perspektif yang ada untuk mendapatkan suatu

informasi dalam data yang sedang dieksplorasi. Hal ini dikarenakan, EDA memiliki karakteristik yang fleksibel sesuai kebutuhan untuk melakukan investigasi dan identifikasi pada suatu fenomena yang muncul saat melakukan penelitian empiris (Wahyuni et al., 2019).

Dengan melakukan EDA, kita dapat memahami kondisi *dataset* yang akan digunakan dalam penelitian. Memahami kondisi *dataset* yang akan dilakukan dengan berbagai cara, diantaranya dengan mengekstrak variabel penting dan meninggalkan variabel yang tidak berguna, mengidentifikasi nilai yang hilang atau potensi kesalahan manusia pada saat memasukkan data, dan memahami hubungan atau kekurangan antar variabel, dan memaksimalkan wawasan yang dimiliki atas kondisi data serta mengurangi potensi kesalahan sebelum data diolah lebih lanjut.

Hubungan antar variabel dapat berbentuk hubungan negatif dan hubungan positif. Korelasi yang positif terjadi ketika variabel X mengalami kenaikan dan diikuti dengan variabel Y. Sementara korelasi negatif terjadi ketika variabel X mengalami kenaikan sementara variabel Y mengalami penurunan. Beberapa metode yang sering digunakan dalam mengukur relasi, yaitu koefisien korelasi pearson, koefisien korelasi spearman, dan koefisien data berkelompok.

EDA menjadi langkah krusial dalam pemodelan *machine learning* karena tanpa melakukan EDA maka kemungkinan hilangnya informasi penting dalam *dataset* dapat terjadi. Jika pemodelan data dilakukan tanpa menerapkan EDA, ada kemungkinan model yang telah dibuat memiliki kinerja yang buruk dikarenakan pemodelan dilakukan tanpa memahami keadaan data yang akan diteliti. Hal ini dapat menyebabkan banyak waktu terbuang hanya untuk mencari kesalahan mana yang perlu diperbaiki dan mengulang proses pemodelan data.

Ada beberapa teknik dalam melakukan EDA, diantaranya:

2.4.1. Statistik Deskriptif

Statistik deskriptif digunakan untuk mendeskripsikan atau merangkum data sehingga menghasilkan informasi secara umum tanpa bertujuan untuk menarik kesimpulan. Informasi yang dapat disajikan oleh statistik deskriptif seperti nilai rata-rata, median, dan modus, yang menunjukkan titik pusat data. Ukuran dispersi

seperti standar deviasi dan variasi yang mengukur sebaran atau penyimpangan data dari nilai rata-ratanya. Statistik deskriptif memungkinkan peneliti untuk melihat gambaran awal data, mengidentifikasi potensi anomali pada data atau *outlier*, serta mempersiapkan data untuk dianalisis lebih mendalam pada tahapan selanjutnya.

Statistik deskriptif dapat disajikan dalam berbagai visualisasi yang mudah dipahami. Visualisasi yang sering digunakan dalam statistik deskriptif seperti tabel, diagram, dan grafik. Visualisasi ini disajikan untuk memberi gambaran awal yang jelas mengenai distribusi dan ringkasan pada data yang akan diproses lebih lanjut. Dengan adanya visualisasi ini juga, dapat membantu peneliti untuk adanya anomali atau *outlier* yang telah disebutkan.

2.4.2. *Univariate Analysis*

Analisis *univariate* merupakan teknik EDA yang berfokus pada pemeriksaan dan deskripsi satu variabel atau kolom data secara terpisah. Analisis *univariate* dilakukan untuk menganalisis dan memahami distribusi data dalam suatu variabel seperti nilai rata-rata, sebaran, dan frekuensi variabel. Analisis ini dapat dibagi menjadi dua yang dapat dilakukan pada data numerik yang disebut analisis kuantitatif dan data kategorial yang disebut analisis kualitatif.

Analisis data numerik melibatkan visualisasi seperti histogram atau *box plot* untuk menganalisis pola distribusi pada data untuk mengetahui jenis distribusi baik distribusi normal maupun distribusi positif dan negatif. Sedangkan, analisis kualitatif menggunakan *bar chart* untuk menampilkan frekuensi atau proporsi setiap kategori. Hasil analisis ini menjadi dasar untuk mengambil keputusan awal mengenai data yang akan dianalisis dan dapat membantu dalam menentukan jenis analisis inferensial yang paling cocok untuk digunakan selanjutnya.

2.4.3. *Multivariate Analysis*

Analisis multivariat merupakan teknik analisis data untuk menganalisis hubungan antara dua atau lebih variabel secara bersamaan. Teknik ini memungkinkan peneliti untuk mengungkap pola dan interaksi yang lebih kompleks pada data dari berbagai fitur dalam *dataset*. Analisis ini dilakukan untuk mengidentifikasi bagaimana satu variabel dapat memengaruhi atau berhubungan

dengan beberapa variabel lain yang dapat membantu dalam membangun model prediksi yang lebih akurat.

Salah satu contoh yang sering digunakan dalam analisis multivariat adalah analisis korelasi, di mana satu variabel dibandingkan dengan beberapa variabel lain untuk mengukur hubungan antara variabel-variabel tersebut. Korelasi antar variabel dapat bersifat positif di mana jika satu variabel meningkat, maka variabel lain ikut meningkat, atau korelasi negatif, di mana jika satu variabel meningkat, maka variabel lain menurun atau sebaliknya.

2.5. Data Pre-Processing

Data Pre-processing berperan penting dalam pengolahan data. *Data pre-processing* merupakan sebuah konsep untuk mengubah data mentah atau data kotor menjadi data bersih yang siap untuk diolah. Data-data yang ada pada kehidupan sehari-hari umumnya memiliki beberapa masalah, seperti data yang atributnya tidak lengkap dan tidak merepresentasikan data yang ada, atau adanya nilai yang hilang pada beberapa atribut atau pada sampel tertentu. Apabila data yang ada sudah cukup untuk merepresentasikan kebutuhan untuk pemodelan maka sampel data yang tidak lengkap bisa dihilangkan. Akan tetapi, jika data yang dimiliki terlalu sedikit atau ingin tetap mempertahankan sampel tersebut, dibutuhkan suatu teknik atau algoritma untuk melengkapi kekurangan data tersebut (Sari, 2018).

Banyaknya *noise* atau *dataset* yang memiliki kesalahan atau berupa *outlier* pada data, sehingga membutuhkan suatu teknik untuk menyaring data-data penting dan memisahkan *outlier* tersebut. *Outlier* merupakan suatu data yang terlalu jauh dan mempunyai perbedaan yang signifikan dibandingkan dengan data lainnya pada *dataset*. Hal ini dapat menyebabkan penggunaan data menjadi bias karena data ini tidak dapat mencerminkan data yang dibutuhkan. Namun, ada kalanya *outlier* berperan penting dalam beberapa penelitian atau analisis suatu permasalahan diantaranya untuk mendiagnosis suatu penyakit pada tanaman, mendeteksi kerugian suatu perusahaan dan lain-lain (Sari, 2018). Data yang tidak konsisten juga perlu ditangani, data ini berupa data yang nilai pada atributnya tidak sama atau tidak seragam yang dapat menyebabkan data yang ada memiliki kesan yang kurang jelas.

Dalam *data pre-processing* juga dilakukan transformasi data seperti:

2.5.1. Deteksi dan Penanganan *Outlier*

Outlier merupakan data yang memiliki nilai jauh berbeda dari sebagian besar data lainnya. Keberadaan *outlier* dapat menyebabkan distorsi pada distribusi data, menurunkan performa model, serta meningkatkan risiko *overfitting*, terutama pada algoritma yang sensitif terhadap jarak atau distribusi data (Aggarwal, 2017). Dalam konteks penelitian ini, *outlier* dapat muncul pada variabel numerik seperti *Learning Time*, *Daily Internet Usage*, atau skor dimensi belajar daring. Oleh karena itu, dilakukan deteksi dan penanganan *outlier* menggunakan pendekatan *Interquartile Range* (IQR). Metode IQR bekerja dengan menghitung nilai kuartil pertama (Q1) dan kuartil ketiga (Q3) dari suatu variabel, lalu menentukan rentang $IQR = Q3 - Q1$. Nilai yang berada di luar batas bawah dan atas berikut dianggap sebagai *outlier*:

$$\text{Batas bawah} = Q1 - 1,5 \times IQR$$

$$\text{Batas atas} = Q3 + 1,5 \times IQR$$

Proses ini membantu mengurangi variabilitas data ekstrem yang dapat mengganggu pembelajaran model, sehingga algoritma seperti *Logistic Regression* dan *Random Forest* dapat berjalan dengan lebih efisien.

2.5.2. *Data Encoding*

Encoding data kategori adalah sebuah proses untuk mengonversi data kategorik menjadi data numerik agar dapat diolah pada algoritma *machine learning*. Metode yang digunakan untuk melakukan *encode* pada data kategori menyesuaikan dengan skala pengukuran data kategori itu sendiri. Ada dua jenis skala pengukuran data kategori yaitu skala ordinal dan skala nominal. Data dengan skala ordinal adalah data kategori yang dapat dibedakan dan dapat diurutkan tingkatannya seperti data tingkat kepuasan pelanggan yang dinyatakan dengan sangat tidak puas, tidak puas, cukup puas, puas, dan sangat puas. sementara untuk data skala nominal merupakan jenis data kategori yang dipergunakan hanya untuk penggolongan saja dan tidak dapat diurutkan tingkatannya sebagai contoh, data jenis kelamin laki-laki dan perempuan (Ramdani & Utami, 2022).

Untuk melakukan *encoding* pada data kategori ordinal dilakukan dengan cara mengurutkan sesuai dengan tingkatannya lalu diberi penomoran sesuai tingkatannya. Sebagai contoh, sangat tidak puas (1), tidak puas (2), cukup puas (3), puas (4), sangat puas (5). Sementara untuk melakukan *encode* skala nominal

dilakukan dengan metode *one-hot-encoding*. Metode ini akan mengubah data kategori menjadi data biner 0 dan 1, di mana semua elemen nantinya akan bernilai 0 terkecuali elemen yang memiliki kategori tersebut akan bernilai 1.

2.5.3. *Feature Scaling*

Feature scaling pada data numerik dilakukan karena pada beberapa *dataset* terdapat perbedaan rentang nilai di setiap atributnya. Perbedaan rentang nilai pada setiap atribut menyebabkan tidak berfungsinya atribut yang memiliki rata-rata jauh lebih kecil dibandingkan dengan atribut-atribut lainnya. Oleh karena itu diperlukan adanya *feature scaling* data numerik untuk menyamakan rentang nilai pada setiap atribut dengan skala tertentu (Nasution et al., 2019). Ada beberapa metode yang sering digunakan untuk *feature scaling* data numerik diantaranya adalah normalisasi *min-max* dan normalisasi *z-score*.

Normalisasi *min-max* akan mengubah data numerik menjadi skala 0 hingga 1. Rumus untuk normalisasi *min-max* adalah sebagai berikut:

$$X_{norm} = \frac{X_{old} - X_{min}}{X_{max} - X_{min}}$$

Di mana:

X_{norm} : Nilai x baru setelah normalisasi

X_{old} : Nilai x lama

X_{min} : Nilai minimum dari fitur

X_{max} : Nilai maksimum dari fitur

Sementara untuk normalisasi *z-score* adalah normalisasi berdasarkan nilai rata-rata dan standar deviasi. Rumus normalisasi *z-score* adalah sebagai berikut:

$$X_{norm} = \frac{X_{old} - \mu}{\sigma}$$

Di mana:

X_{norm} : Nilai x baru setelah dinormalisasi

X_{old} : Nilai x lama

μ : Nilai rata-rata populasi

σ : Nilai simpangan baku

2.5.4. *Balancing Data*

Selain *encoding* dan *feature scaling*, pada beberapa kasus *dataset* juga dilakukan *balancing data* untuk menyeimbangkan kelas atau label pada *dataset*. Menyeimbangkan kelas pada *dataset* dilakukan agar model *machine learning* tidak memiliki kinerja yang buruk karena proses klasifikasinya hanya dilakukan pada kelas atau label yang mayoritas (Ardiningtyas & Rosa, 2021). Salah satu metode untuk menyeimbangkan data yang tidak seimbang dalam menggunakan metode *Synthetic Minority Oversampling Technique* (SMOTE). SMOTE pertama kali diperkenalkan oleh Nithes V Chawla dengan cara membuat replikasi data yang biasa dikenal dengan data sintesis. Metode SMOTE bekerja mencari *K-Nearest Neighbor* (KNN) untuk setiap data pada kelas minoritas, lalu dibuat data sintesis sebanyak persentase duplikasi yang diinginkan antara data minor dan KNN yang dipilih secara acak (Siringoringo, 2018).

2.6. *Synthetic Minority Oversampling Method*

Ketidak seimbangan data terjadi ketika jumlah data pada satu kelas jauh lebih banyak dibandingkan kelas lainnya, sehingga model klasifikasi menjadi bias terhadap kelas lainnya, sehingga model klasifikasi menjadi bias terhadap kelas mayoritas (He & Gracia, 2009). Dalam penelitian ini, fenomena tersebut muncul karena jumlah mahasiswa dengan IPK tinggi jauh lebih banyak dibandingkan mahasiswa dengan IPK rendah. Kondisi ini menyebabkan model sulit mempelajari pola dari mahasiswa berprestasi rendah, yang tercermin pada rendahnya nilai *recall* dan *F1-Score* pada kelas tersebut. Untuk mengatasi hal tersebut, penelitian ini menggunakan SMOTE (*Synthetic Minority Oversampling Method*), yaitu metode *oversampling* yang dikembangkan oleh Chawla et al. (2002). Berbeda dengan *Random Oversampling* yang hanya menggandakan data minoritas, SMOTE menciptakan data sintesis baru melalui interpolasi linier antara sampel minoritas yang berdekatan. Dengan demikian, distribusi kelas menjadi lebih seimbang tanpa menambah data duplikat.

Secara umum, proses kerja SMOTE dimulai dengan menentukan jumlah tetangga terdekat (*K-nearest neighbors*) untuk setiap data minoritas dengan nilai $k=5$. Selanjutnya memilih salah satu tetangga terdekat secara acak dan membentuk data sintesis baru di antara dua titik data minoritas berdasarkan perhitungan :

$$x_{new} = x_i + \lambda(x_{zi} - x_i)$$

Di mana:

x_i : Titik data minoritas awal.

x_{zi} : Tetangga terdekat x_i .

λ : Bilangan acak antara 0 dan 1.

Dengan cara ini, SMOTE memperluas ruang representasi kelas minoritas tanpa menghapus data mayoritas, sehingga model dapat mengenali pola dari kelas IPK rendah dengan lebih akurat. Penerapan SMOTE pada penelitian ini dilakukan pada data latih setelah proses *encoding* dan normalisasi data, dan sebelum pelatihan model dimulai. SMOTE dipilih karena teknik ini terbukti meningkatkan performa model dalam mengklasifikasikan kelas minoritas, sebagaimana juga dijelaskan oleh Fernández et al. (2018), bahwa SMOTE efektif untuk meningkatkan *recall* dan *F1-Score* pada *dataset* dengan ketidakseimbangan tinggi.

2.7. Machine Learning

Machine learning (ML) atau pembelajaran mesin merupakan salah satu cabang penting dari Kecerdasan Buatan (*Artificial Intelligence* - AI) yang berfokus pada pengembangan sistem komputasi yang mampu belajar secara mandiri dari data tanpa perlu diprogram secara eksplisit untuk setiap tugas spesifik (Samuel, 1959). Inti dari ML adalah bagaimana mesin dapat "belajar" dengan menganalisis sekumpulan data (*dataset*), mengidentifikasi pola-pola tersembunyi di dalamnya, dan kemudian menggunakan pola tersebut untuk membuat keputusan, klasifikasi, atau prediksi terhadap data baru yang belum pernah dilihat sebelumnya (Bishop,

2006). Kemampuan ML untuk mengolah data dalam skala besar dan kompleks menjadikannya alat yang sangat berharga di berbagai bidang, termasuk dalam dunia pendidikan untuk menganalisis faktor-faktor yang mempengaruhi proses belajar mengajar, seperti yang akan dilakukan dalam penelitian ini.

Secara umum, terdapat beberapa paradigma utama dalam *Machine Learning*, yaitu *Supervised Learning* (Pembelajaran Terarah/Terawasi), *Unsupervised Learning* (Pembelajaran Tak Terarah/Tak Terawasi), dan *Reinforcement Learning* (Pembelajaran Penguatan) (Murphy, 2012). *Unsupervised Learning* bertujuan menemukan struktur atau pola tersembunyi dalam data tanpa adanya label target (misalnya, pengelompokan mahasiswa berdasarkan gaya belajar). *Reinforcement Learning* fokus pada bagaimana agen belajar mengambil keputusan optimal melalui interaksi dengan lingkungannya berdasarkan sistem *reward* dan *punishment*. Namun, paradigma yang paling relevan untuk tujuan penelitian ini adalah *Supervised Learning*.

2.7.1. *Supervised Learning*

Supervised learning adalah jenis pembelajaran mesin di mana algoritma belajar dari *dataset* yang sudah memiliki "label" atau "jawaban" yang benar (Mohri et al., 2018). Dalam konteks ini, setiap data masukan (*input*) dalam *dataset* pelatihan memiliki pasangan keluaran (*output*) atau target yang sudah diketahui. Data masukan ini terdiri dari serangkaian *features* atau variabel independen (X), sedangkan data keluaran adalah variabel dependen atau label target (Y) yang ingin diprediksi oleh model. Tujuan utama dari *Supervised Learning* adalah untuk mempelajari sebuah fungsi pemetaan (f) yang dapat memodelkan hubungan antara *input* (X) dan *output* (Y), sehingga $Y = f(X)$. Proses "belajar" terjadi karena algoritma secara iteratif membandingkan hasil prediksinya dengan label target yang sebenarnya pada data pelatihan, kemudian menyesuaikan parameter internalnya untuk meminimalkan kesalahan (*error*) prediksi.

Terdapat dua jenis tugas utama dalam *Supervised Learning* (Hastie et al., 2009), Pertama Klasifikasi (*Classification*), Digunakan ketika variabel target (*output*) bersifat kategorial atau diskrit. Tujuannya adalah untuk memprediksi kelas

atau kategori dari data baru berdasarkan fitur-fiturnya. Contoh dalam konteks pendidikan adalah memprediksi apakah seorang mahasiswa akan 'Lulus Tepat Waktu' atau 'Tidak Lulus Tepat Waktu', atau mengklasifikasikan tingkat risiko akademik ('Rendah', 'Sedang', 'Tinggi'). Algoritma klasifikasi yang umum digunakan antara lain *Logistic Regression*, *Support Vector Machine* (SVM), *Decision Tree*, *Naïve Bayes*, dan *Random Forest* (dalam mode klasifikasi).

Regresi (*Regression*) Digunakan ketika variabel target (*output*) bersifat kontinu atau numerik. Tujuannya adalah untuk memprediksi nilai numerik dari data baru. Contoh dalam konteks pendidikan adalah memprediksi nilai Indeks Prestasi Kumulatif (IPK) mahasiswa di akhir semester atau memprediksi skor ujian berdasarkan waktu belajar dan faktor lainnya. Algoritma regresi yang umum digunakan meliputi *Linear Regression*, *Decision tree*, *Support Vector Regression* (SVR), dan *Random Forest* (dalam model regresi).

2.7.2. *Unsupervised Learning*

Selain *Supervised Learning*, paradigma penting lainnya dalam *Machine Learning* adalah *Unsupervised Learning* atau Pembelajaran Tak Terarah/Tak Terawasi. Perbedaan fundamental antara keduanya terletak pada jenis data yang digunakan selama proses pelatihan. Dalam *Unsupervised Learning*, algoritma "belajar" dari *dataset* yang tidak memiliki label target atau output yang telah ditentukan sebelumnya (Murphy, 2012). Tujuan utama dari *Unsupervised Learning* adalah untuk menemukan struktur, pola, atau hubungan tersembunyi yang menarik secara intrinsik dalam data itu sendiri, tanpa panduan dari variabel *output* yang diketahui.

Algoritma tidak diberi tahu "jawaban yang benar", melainkan harus mengeksplorasi data untuk mengungkap wawasan atau representasi data yang lebih ringkas. Karena tidak ada label untuk memvalidasi hasil secara langsung, evaluasi model *Unsupervised Learning* sering kali lebih kualitatif atau bergantung pada metrik yang mengukur properti dari struktur yang ditemukan.

Unsupervised learning memiliki dua tugas utama, yang pertama Mengelompokkan sekumpulan *instance* data ke dalam beberapa grup (klaster)

sedemikian rupa sehingga *instance* dalam satu klaster memiliki kemiripan yang tinggi satu sama lain, sementara *instance* di klaster yang berbeda memiliki kemiripan yang rendah (James et al., 2013). Kemiripan biasanya diukur berdasarkan nilai-nilai fitur dari *instance* data. Kedua, yaitu mengurangi jumlah variabel atau fitur dalam *dataset* sambil tetap mempertahankan informasi penting atau struktur data yang esensial. Pengurangan dimensi berguna untuk mengatasi *curse of dimensionality*, mengurangi kompleksitas komputasi, menghilangkan redundansi fitur, dan memfasilitasi visualisasi data.

2.7.3. *Reinforcement Learning*

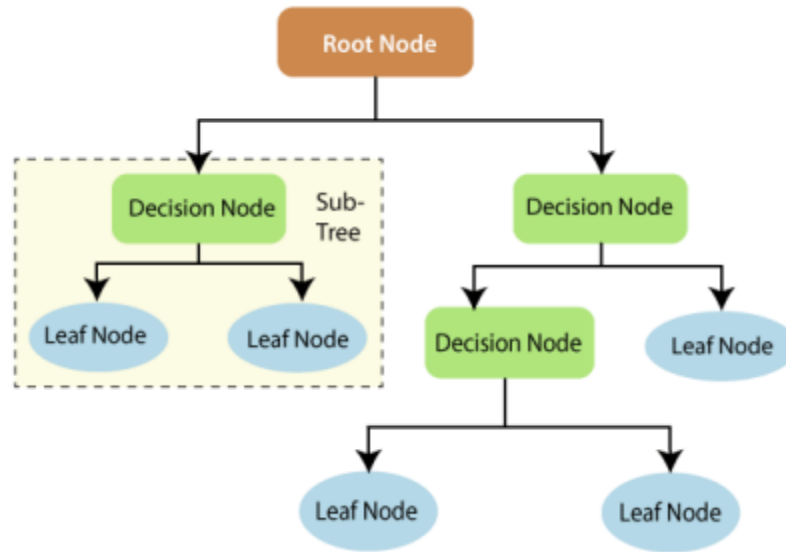
Selain *Supervised Learning* dan *Unsupervised Learning*, paradigma ketiga dalam *Machine Learning* adalah *Reinforcement Learning* (RL) atau Pembelajaran Penguatan. Berbeda dengan dua paradigma lainnya, RL berfokus pada bagaimana seorang agen (*agent*) belajar untuk mengambil serangkaian keputusan atau tindakan (*actions*) dalam suatu lingkungan (*environment*) untuk memaksimalkan imbalan (*reward*) kumulatif jangka panjang (Sutton & Barto, 2018).

Dalam *Reinforcement Learning*, agen belajar melalui proses interaksi coba-dan-gagal (*trial-and-error*). Agen mengamati keadaan (*state*) saat ini dari lingkungan. Berdasarkan keadaan yang ada, agen memilih dan melakukan suatu tindakan. Lingkungan merespons tindakan agen dengan bertransisi ke keadaan baru dan memberikan imbalan (*reward*) atau hukuman (*punishment*) kepada agen. Imbalan bisa positif untuk tindakan yang baik atau negatif untuk tindakan yang buruk. Agen menggunakan informasi dari imbalan dan keadaan baru ini untuk memperbaharui pengetahuannya atau kebijakannya (*policy*), yaitu strategi untuk memilih tindakan di masa depan.

2.8. *Decision Tree*

Decision tree yang memiliki arti pohon keputusan adalah struktur yang menyerupai diagram alir (*flow chart*), di mana setiap simpul internal (*non-leaf node*) menandakan sebuah pengujian pada sebuah atribut, setiap cabang merepresentasikan sebuah hasil dari pengujian tersebut, dan setiap simpul daun (*leaf*

node atau *terminal node*) menyimpan sebuah label kelas. Simpul paling atas dalam sebuah pohon adalah simpul akar (*root node*) (Han, J., Kamber, M., & Pei, J. 2012).



Gambar 2. 1 Decision Tree (Islam Alharrem, 2025)

Berikut adalah penjelasan komponen-komponen dalam *decision tree*:

Tabel 2. 1. Elemen Decision Tree

Elemen <i>Decision Tree</i>	Deskripsi	Fungsi
<i>Root node</i>	Simpul titik awal dalam <i>decision tree</i>	Mewakili seluruh populasi <i>dataset</i> yang akan dibagi. Dari simpul ini, pengujian awal terhadap atribut dilakukan untuk memulai proses pemisahan data
<i>Decision node</i>	Simpul yang berada diantara <i>root node</i> dan <i>leaf node</i> .	Merepresentasikan keputusan berdasarkan nilai dari sebuah atribut atau fitur tertentu. Tujuannya untuk

Elemen <i>Decision Tree</i>	Deskripsi	Fungsi
		membagi data yang masuk ke simpul tersebut menjadi himpunan bagian yang lebih homogen.
<i>Leaf node</i>	Simpul-simpul yang berada di bagian paling bawah dari setiap jalur dalam pohon dan tidak memiliki cabang keluar.	Merepresentasikan hasil akhir atau keputusan /prediksi. Setiap simpul ini menandakan label kelas atau nilai kontinu untuk semua data yang mencapai simpul tersebut.
<i>Parent node</i>	Simpul yang dibagi menjadi beberapa simpul di bawahnya	Setiap <i>decision node</i> adalah parent node bagi simpul-simpul yang langsung berada di bawahnya
<i>Child node</i>	Simpul yang dihasilkan dari pembagian sebuah <i>parent node</i>	Mewakili salah satu hasil tes atau keputusan yang dilakukan parent node. Simpul ini dapat menjadi <i>decision node</i> atau <i>leaf node</i>
<i>Branch</i>	Garis-garis yang menghubungkan setiap simpul.	Merepresentasikan hasil atau konsekuensi dari pengujian yang dilakukan pada <i>decision node</i> . Tiap cabang biasanya diberi label sesuai dengan nilai atau rentang nilai dari atribut yang diuji, yang mengarahkan data ke simpul berikutnya.

Elemen <i>Decision Tree</i>	Deskripsi	Fungsi
<i>Sub-tree</i>	Bagian dari pohon keputusan yang juga memiliki struktur pohon	Menunjukkan sifat rekursif dari pembentukan pohon keputusan, di mana setiap <i>decision node</i> dapat dianggap sebagai <i>root node</i> dari sebuah <i>sub-tree</i> yang lebih kecil
<i>Splitting</i> (pemisahan)	Proses membagi himpunan data pada suatu simpul baik pada <i>root node</i> atau <i>decision node</i> menjadi beberapa himpunan bagian berdasarkan kriteria tertentu dari suatu atribut.	Mengurangi <i>impurity</i> atau varians dalam subset data, sehingga menghasilkan kelompok-kelompok yang lebih murni atau homogen berdasarkan variabel target.
<i>Pruning</i> (pemangkasan)	Teknik yang digunakan untuk mengurangi ukuran pohon keputusan dengan menghilangkan bagian-bagian yang dianggap kurang memberikan informasi prediktif atau berpotensi menyebabkan <i>overfitting</i> .	Meningkatkan kemampuan generalisasi model pada data baru yang belum pernah dilihat dan mengurangi kompleksitas pohon.

Proses pembentukan pohon-pohon keputusan pada dasarnya adalah proses *Recursive partitioning*. Proses ini dimulai dari simpul akar yang berisi *dataset* yang akan dilatih, algoritma secara berulang memilih atribut terbaik untuk membagi data

menjadi himpunan bagian yang lebih kecil dan lebih homogen berdasarkan variabel target. Proses ini berlanjut hingga kondisi berhenti terpenuhi.

Pada setiap simpul, algoritma mengevaluasi semua kemungkinan atribut dan titik *splitting* atau nilai kategori untuk menemukan *splitting* terbaik. *Splitting* dapat dikatakan baik diukur berdasarkan seberapa murni atau homogen himpunan bagian yang dihasilkan setelah proses *splitting*. Beberapa kriteria pemisahan yang umum digunakan untuk tugas klasifikasi adalah *Gini Impurity*.

Gini impurity mengukur tingkat tidak murnian atau probabilitas kesalahan klasifikasi suatu sampel jika labelnya ditentukan secara acak berdasarkan distribusi kelas pada simpul tersebut. Nilai *gini* berkisar antara 0 yang menandakan simpul murni di mana semua sampel milik satu kelas hingga 0,5 di mana distribusi kelas merata. Algoritma akan memilih atribut dan titik pemisahan yang menghasilkan penurunan *Gini impurity* terbesar. Rumus *Gini impurity* untuk satu simpul t adalah sebagai berikut:

$$Gini(t) = 1 - \sum_{j=1}^k [P(j|t)]^2$$

$Gini(t)$: Nilai *Gini impurity* untuk sebuah simpul spesifik yang disebut sebagai simpul t

t : Mewakili simpul tertentu dalam pohon keputusan yang sedang dievaluasi tingkat ketidakmurniannya.

$P(j|t)$: Proporsi dari sampel data yang termasuk dalam kelas j dan simpul t

k : Jumlah kelas pada variabel target

Proses *splitting* rekursif dapat dikatakan selesai apabila salah satu atau lebih kondisi berikut terpenuhi, diantaranya semua sampel pada simpul saat ini memiliki kelas yang sama atau tidak ada atribut lagi yang dapat digunakan untuk membagi data.

2.9. *Random Forest*

Random Forest pertama kali dikenalkan oleh Leo Breiman pada tahun 2001 melalui karyanya yang berjudul “*Random Forest*”. Metode ini merupakan evolusi dari *decision tree*, yang menggabungkan beberapa pohon keputusan. Setiap pohon dalam ansambel ini bergantung pada nilai-nilai vektor sampel yang acak dan independen, serta distribusi yang sama pada semua pohon yang ada dalam kelompok hutan tersebut (Santra & Christy, 2012). Metode pengajaran *decision tree* merupakan salah satu teknik klasifikasi yang sangat diminati dikarenakan tingkat akurasi yang diberikan sebanding dengan teknik klasifikasi lainnya, juga memiliki efisiensi yang tinggi. Model klasifikasi yang dihasilkan melalui teknik ini direpresentasikan sebagai pohon dan dikenal dengan istilah *Random Forest*. Beberapa algoritma pembelajaran *decision tree* yang cukup terkenal diantaranya ID3 dan C4.5 oleh Quinlan (1986), serta CART (*Classification and Regression Trees*) oleh Breiman, Friedman, Stone, dan Olshen (1984).

CART merupakan metode pemodelan yang memisahkan data secara rekursif menjadi subset yang semakin homogen berdasarkan atribut yang paling informatif. CART dapat digunakan baik untuk klasifikasi maupun regresi, dengan proses pemilahan didasarkan pada ketidakmurnian seperti *Gini Index* (Han et al., 2012). Dalam *Random Forest*, pemrosesan data dilakukan dengan cara yang serupa dengan yang dilakukan pada CART tanpa adanya tahap pemangkasan (*pruning*). *Gini Index* digunakan untuk memilih fitur di setiap simpul internal dari *Decision Tree*. *Gini Index* merupakan nilai untuk mengukur penurunan *Gini Impurity* yang terjadi setelah suatu atribut digunakan untuk memisahkan data. Semakin besar penurunan nilai ketidakmurnian, semakin baik kualitas pemisahan tersebut (Breiman et al., 1984).

Gini Index untuk suatu pembelahan s pada *parent node* t dapat dinyatakan sebagai:

$$GiniIndex(t, s) = Gini(t) - \sum_{i=1}^m \frac{N_i}{N} \times Gini(t_i)$$

di mana:

$Gini(t)$: Nilai *Gini impurity parent node* sebelum pembelahan

$Gini(t_i)$: Nilai *impurity child node* ke- i

N_i : Jumlah data pada *child node* ke- i

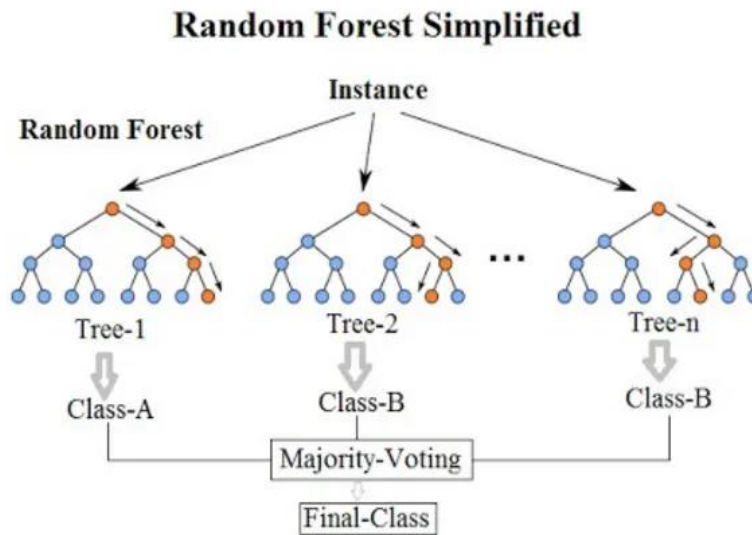
N_t : Jumlah data pada *parent node*

m : Jumlah *child node* hasil pembelahan

Tingkat kepercayaan terhadap model dapat dinilai menggunakan *Margin Function*. *Margin Function* digunakan untuk mengevaluasi stabilitas dan keandalan prediksi dalam algoritma *Random forest*. Secara matematis, margin untuk suatu pasangan data (X, Y) didefinisikan sebagai berikut:

$$mr(X, Y) = P_{\theta}(h(X, \theta) = Y) - \max_{j \neq Y} P_{\theta}(h(X, \theta) = j)$$

Di mana $h(X, \theta)$ merepresentasikan prediksi dari satu pohon dalam *forest* yang dibangun dengan parameter acak θ , sedangkan P_{θ} menunjukkan probabilitas agregat seluruh variasi acak dari parameter tersebut. Nilai margin positif menandakan bahwa mayoritas pohon dalam *forest* memprediksi kelas yang benar dengan tingkat kepercayaan lebih tinggi dibandingkan kelas lainnya, sehingga prediksi dianggap stabil dan akurat. Sebaliknya, margin negatif menunjukkan bahwa model memiliki ketidakpastian dalam menentukan kelas sebenarnya. Rata-rata margin keseluruhan disebut dengan *expected margin* dan digunakan untuk memperkirakan *generalization error* dari model. Semakin besar nilai margin rata-rata, semakin kecil kemungkinan model melakukan kesalahan pada data baru (Breiman, 2001).



Gambar 2. 2. Random Forest (Wei, 2022)

Secara umum, algoritma *Random Forest* meliputi beberapa tahap. Pertama, algoritma mengambil sejumlah sampel data acak menggunakan teknik *bootstrap sampling* dari *dataset* asli. Kedua, untuk setiap sampel *bootstrap*, dibangun satu *Decision Tree* yang dilatih secara independen. Ketiga, proses prediksi dilakukan dengan menggabungkan hasil dari seluruh pohon menggunakan metode *majority voting* untuk klasifikasi atau *averaging* untuk regresi. Hasil akhir merupakan keputusan dengan jumlah suara terbanyak atau rata-rata tertinggi dari semua pohon dalam ansambel (Trivusi, 2022).

2.10. Logistic Regression

Metode linear merupakan salah satu kelas algoritma fundamental dalam statistika dan *machine learning* yang telah lama menjadi dasar bagi banyak proses analisis data. Pendekatan ini bertujuan untuk memodelkan hubungan antara satu variabel dependen (variabel target) dengan satu atau lebih variabel independen (variabel prediktor) dengan mengasumsikan adanya hubungan linear antara keduanya (Tibishirani dan Freidman, 2019). Meskipun sederhana, metode linear sangat banyak digunakan, baik sebagai model mandiri maupun sebagai dasar untuk teknik yang lebih canggih.

Regresi linear adalah teknik statistik yang paling dasar dan banyak digunakan untuk menyelidiki hubungan linear antara variabel dependen yang bersifat kontinu dan satu atau lebih variabel independen (James et al., 2013). Tujuan utama dari regresi linear adalah untuk membangun model statistik yang dapat digunakan untuk memprediksi nilai variabel dependen berdasarkan nilai variabel independen yang diketahui, serta untuk memahami kontribusi masing-masing variabel independen (Kutner et al., 2015). Terdapat dua jenis utama regresi linear:

2.10.1. Regresi Linear Sederhana

Regresi linear sederhana digunakan ketika hanya ada satu variabel independen untuk memprediksi variabel dependen. Model matematikanya sebagai berikut:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Di mana:

- Y : Variabel dependen
- X : Variabel independen
- β_0 : Nilai Intersep, merupakan nilai perkiraan Y ketika $X = 0$
- β_1 : Koefisien regresi
- ϵ : *Error term*

2.10.2. Regresi Linear Berganda

Regresi linear berganda digunakan ketika terdapat dua atau lebih variabel independen untuk memprediksi variabel dependen. Model matematisnya adalah sebagai berikut:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \epsilon$$

Di mana:

- Y : Variabel dependen
- $X_1, X_2, \dots, X_p$: Variabel independen
- β_0 : Nilai Intersep
- $\beta_1, \beta_2, \dots, \beta_p$: Koefisien regresi
- ϵ : *Error term*

2.10.3. Logistic Regression

Sementara Regresi Linear bertujuan untuk memprediksi variabel dependen yang bersifat kontinu, Regresi Logistik adalah metode linear yang digunakan ketika variabel dependen bersifat kategorikal. Hal ini menjadikannya pilihan yang tepat untuk tugas klasifikasi, seperti memprediksi apakah seorang mahasiswa termasuk dalam kategori IPK "Tinggi", "Sedang", atau "Rendah". Meskipun namanya mengandung kata "regresi," *output* utama dari Regresi Logistik adalah probabilitas bahwa suatu *instance* data termasuk dalam kelas tertentu.

Regresi logistik memiliki tujuan untuk memodelkan probabilitas dari sebuah *event* atau keanggotaan dalam suatu kelas berdasarkan satu atau lebih variabel prediktor, juga untuk mengklasifikasikan observasi ke dalam kategori-kategori yang telah ditentukan berdasarkan probabilitas yang diprediksi. Berbeda dengan regresi linear yang memodelkan Y secara langsung, regresi logistik memodelkan probabilitas $P(Y = 1|X)$. Karena probabilitas harus berada di antara 0 dan 1, kombinasi linear dari fitur input tidak bisa langsung digunakan sebagai *output*. Sebaliknya, nilai z ditransformasikan menggunakan fungsi logistik atau fungsi *sigmoid*:

$$P(X) = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$

Fungsi *sigmoid* ini selalu menghasilkan *output* antara 0 dan 1 sehingga cocok untuk merepresentasikan probabilitas.

Untuk mendapatkan kembali hubungan linear antara prediktor dan *output* yang ditransformasi, regresi logistik bekerja dengan *log-odds* atau *logit* dari probabilitas:

$$\text{logit}(P) = \ln\left(\frac{P}{1-P}\right) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$$

Di mana rasio $\frac{P}{1-P}$ disebut sebagai *odds*. Dengan transformasi *logit*, didapat model linear pada *log-odds*, yang memungkinkan interpretasi koefisien β_j .

Koefisien dalam regresi logistik tidak diestimasi menggunakan *Ordinary Least Squares* (OLS) karena outputnya bukan kontinu dan asumsi OLS tidak terpenuhi. Namun, parameter diestimasi menggunakan metode *Maximum Likelihood Estimation* (MLE). MLE bertujuan untuk menemukan nilai-nilai parameter yang memaksimalkan probabilitas untuk mengamati data latih yang diberikan dengan model yang diusulkan. Proses ini biasanya melibatkan algoritma optimasi iteratif.

Untuk membuat prediksi kelas aktual, ambang batas diterapkan pada probabilitas yang diprediksi. Jika $P > 0.5$, maka *instance* diklasifikasikan sebagai kelas 1, jika tidak maka diklasifikasikan sebagai kelas 0. Meski *output* probabilitasnya berbentuk non-linear, batas keputusan yang memisahkan kelas-kelas dalam ruang fitur tetap bersifat linear.

Ketika model regresi dihadapkan dengan masalah klasifikasi di mana terdapat lebih dari dua kelas target, sementara klasifikasi dasar hanya dirancang untuk membedakan dua kelas, maka diperlukan sebuah strategi untuk mengadaptasi algoritma biner tersebut. *One-vs-Rest* (OVR) merupakan salah satu strategi dekomposisi yang banyak digunakan dan intuitif untuk mengatasi permasalahan ini. OvR mengubah satu masalah klasifikasi multi-kelas menjadi beberapa masalah klasifikasi biner independen. Jika terdapat K kelas target, maka akan dibangun K model klasifikasi biner yang terpisah.

Implementasi OvR dimulai dengan dekomposisi masalah, di mana untuk setiap kelas C_i akan dibangun sebuah model klasifikasi biner berupa M_i . Model ini dilatih untuk membedakan *instance* dari kelas C_i melawan *instance* dari semua kelas lainnya yaitu kelas $C_1, C_{i+1}, \dots, C_K$. Jika pada data memiliki 3 kategori IPK misal tinggi, sedang, dan rendah, maka akan dilatih 3 model regresi logistik biner yaitu model 1 membandingkan IPK tinggi dengan yang sedang atau rendah, model 2 membandingkan IPK sedang dengan yang tinggi atau rendah, dan model 3 membandingkan IPK rendah dengan tinggi atau sedang.

Setiap model biner M_i dilatih menggunakan seluruh *dataset* latih. Semua *instance* yang termasuk dalam kelas C_i akan diberi label positif, sedangkan *instance*

yang termasuk label kelas lain akan diberi label negatif. Algoritma klasifikasi biner kemudian digunakan untuk melatih masing-masing dari K model ini. Setiap model M_i akan belajar untuk menghasilkan skor atau probabilitas yang menunjukkan seberapa besar kemungkinan sebuah instance baru termasuk dalam kelas C_i dibandingkan dengan semua kelas lainnya.

Ketika sebuah *instance* data baru x_{new} akan diklasifikasikan, instance x_{new} dimasukkan ke dalam semua K model biner ($M_1, M_2, \dots, M_K$). Setiap model akan menghasilkan skor probabilitas. Kelas akhir yang diprediksi untuk x_{new} adalah kelas C_i yang modelnya menghasilkan skor probabilitas tertinggi:

2.11. Hybrid Machine Learning

Hybrid Machine Learning merupakan pendekatan pembelajaran mesin yang mengkombinasikan dua atau lebih algoritma yang memiliki karakteristik berbeda untuk menghasilkan sistem prediksi yang lebih akurat, stabil, dan efisien. Tujuan utama pendekatan *Hybrid Machine Learning* adalah memanfaatkan keunggulan masing-masing metode yang akan digunakan dan meminimalkan kelemahan dari setiap algoritma yang digunakan secara tunggal (Yadav et al., 2023).

Berbeda dengan model konvensional yang hanya menggunakan satu algoritma, *Hybrid Machine Learning* memungkinkan integrasi antar model pada berbagai tingkatan seperti tingkatan data (*data-level*), fitur (*feature-level*), maupun model (*model-level*). Pendekatan ini mencakup proses kolaboratif yang lebih luas dalam sistem pembelajaran mesin. Konsep *hybrid* dapat dipahami sebagai strategi *multi-model intelligence* di mana sistem belajar dari interaksi antar-model. Pendekatan ini telah terbukti meningkatkan performa prediksi di berbagai domain seperti diagnosis penyakit, pertanian, hingga deteksi serangan siber (Haddad et al., 2024).

2.11.1. Kategori Hybrid Machine Learning

Hybrid Machine Learning dapat dibedakan menjadi beberapa kategori berdasarkan titik integrasi antar komponennya (Liao et al., 2025):

1. *Data-Level Hybrid*

Data-level hybrid yaitu menggabungkan data dari beberapa sumber sebelum proses pelatihan model, sebagai contoh yaitu metode *oversampling data* menggunakan SMOTE dan *Decision Tree*

2. *Feature-Level Hybrid*

Feature-level hybrid menggabungkan fitur yang dihasilkan dari metode berbeda, seperti hasil ekstraksi fitur statistik dan fitur hasil transformasi *Deep Learning*.

3. *Model-Level Hybrid*

Model-level hybrid menggabungkan hasil prediksi dari beberapa model melalui metode seperti *stacking*, *bagging*, atau *boosting*.

4. *Algorithmic-Level Hybrid*

Algorithmic-level hybrid mengintegrasikan dua algoritma berbeda ke dalam satu sistem untuk optimasi parameter performa.

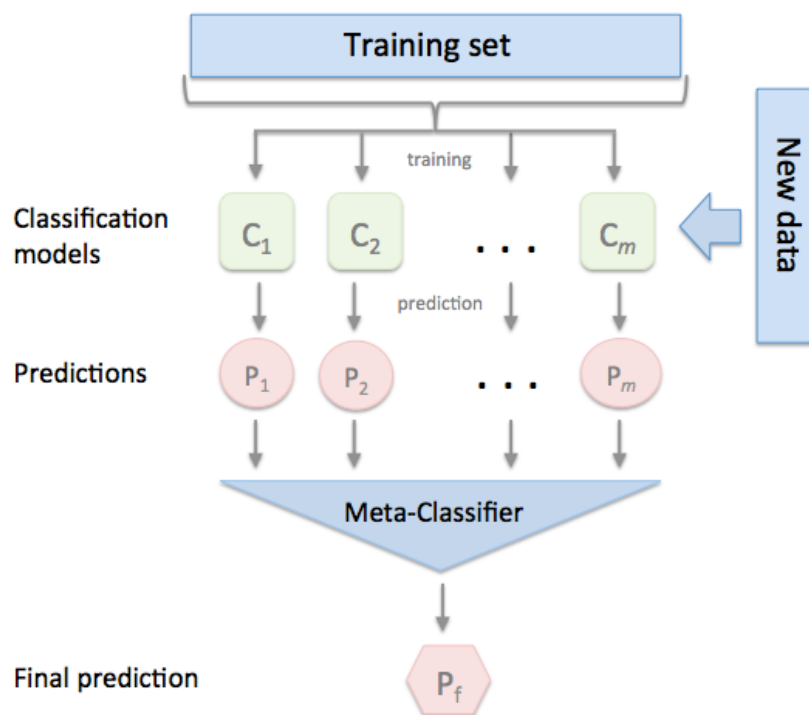
2.11.2. *Model-Level Hybrid*

Model-level hybrid yang dikenal juga sebagai *ensemble stacking*, yaitu pendekatan yang menggabungkan beberapa model pembelajaran dasar (*base model*) untuk menghasilkan model baru (*meta-learner*) yang lebih kuat dan general (Li et al., 2023). Teknik *stacking* pertama kali diperkenalkan oleh Wolpert (1992) dan banyak digunakan untuk meningkatkan akurasi prediksi di berbagai bidang seperti deteksi intrusi (Verma et al., 2024), diagnosis penyakit (Doğru et al., 2023), dan sistem pertanian (Abdallah et al., 2022).

Base model merupakan lapisan pertama dalam arsitektur *stacking*. Model ini berfungsi sebagai *individual learners* yang masing-masing berusaha mempelajari pola atau hubungan antara fitur dan target dari data yang sama (Liao et al., 2025). Dalam metode *stacking* setiap *base model* memiliki karakteristik dan bias algoritmik yang berbeda. Keberagaman karakteristik antar *base model* ini merupakan faktor kunci keberhasilan karena setiap model akan membuat kesalahan yang berbeda, dan

kombinasi kedua model membantu mengurangi varians serta generalisasi (Buyrukoğlu & Savaş, 2023).

Meta-features merupakan *output* dari setiap *base model*, yang berisi prediksi probabilitas terhadap kelas target untuk setiap sampel. Nilai-nilai tidak lagi berupa fitur asli melainkan representasi baru yang menggambarkan tingkat kepercayaan tiap model dasar terhadap suatu sampel (Yadav et al., 2023). Masing-masing model menghasilkan nilai probabilitas p_i untuk setiap sampel x_i . Nilai-nilai tersebut disusun dalam *dataset* baru seperti tabel berikut:



Gambar 2. 3. Dataset Meta Features (Zhang, 2021)

Dataset ini berfungsi untuk menyimpan hasil prediksi dari *base model* yang akan digunakan untuk menggantikan fitur asli untuk tahap pembelajaran berikutnya. Karena setiap model memiliki pola kesalahan berbeda, *meta-features* membantu *meta-learner* belajar untuk mempercayai model tertentu.

Meta-learner merupakan lapisan kedua dari arsitektur *stacking*. Model ini tidak mempelajari fitur asli, melainkan memodelkan hubungan antar *output base model* (*meta-features*). Fungsi utamanya adalah menggabungkan hasil prediksi *base model*

secara adaptif dan optimal untuk menghasilkan keputusan akhir yang lebih akurat (Aslam et al., 2025). Secara formal *meta-learner* mempelajari fungsi berikut:

$$y = g(y_1, y_2, \dots, y_n)$$

Di mana:

$y_1, y_2, \dots, y_n$: Prediksi ke n base model

g : fungsi *meta-learner*

Dalam penelitian ini:

$$y_{final} = f_{meta}([Pred_{RF}, Pred_{LR}])$$

Dengan f_{meta} adalah Regresi logistik yang bertugas untuk menemukan kombinasi bobot terbaik antara hasil prediksi *Random Forest* dan Regresi logistik.

2.12. Evaluasi *Machine Learning*

Setelah sebuah model *machine learning* dibangun, tahapan krusial selanjutnya adalah mengevaluasi kinerja *machine learning* tersebut. evaluasi model bertujuan untuk mengukur seberapa baik model yang telah dilatih dapat melakukan tugas yang diharapkan pada data baru yang belum pernah diproses sebelumnya. Proses evaluasi model ini penting dilakukan untuk memberi ukuran kuantitatif seberapa akurat dan andal model tersebut, memungkinkan perbandingan objektif antara model atau algoritma yang berbeda untuk memilih yang terbaik untuk tugas spesifik, menilai kemampuan model untuk menggeneralisasi pola yang dipelajari dari data latih ke data baru atau data tes, dan membantu mengidentifikasi potensi masalah seperti *overfitting* atau *underfitting*.

Confusion Matrix merupakan salah satu cara untuk mengevaluasi performa suatu model *machine learning*. *Confusion matrix* merupakan tabel kontingensi yang menyajikan ringkasan hasil prediksi model dibandingkan dengan kelas aktual. untuk klasifikasi biner, matriks ini terdiri dari empat elemen. pertama adalah *True Positive* (TP) yang berarti sampel positif yang diklasifikasikan dengan benar, kedua adalah *True Negative* (TN) yang berarti sampel negatif yang diklasifikasikan dengan benar, ketiga yaitu *False Positive* (FP) merupakan sampel negatif atau salah yang

diklasifikasikan benar atau positif, dan terakhir *False Negative* (FN) ketika sampel positif yang salah diklasifikasikan sebagai negatif. Confusion matrix menjadi dasar perhitungan untuk berbagai matriks kinerja lainnya. matriks akurasi menilai proporsi keseluruhan sampel yang diklasifikasikan dengan benar. matriks presisi mengukur proporsi dari sampel yang diprediksi sebagai kelas positif yang benar-benar positif. Matriks *recall* mengukur proporsi dari sampel positif aktual yang berhasil diidentifikasi dengan benar, dan *F1-Score* mendefinisikan rata-rata harmonik dari presisi dan *recall*, memberi satu ukuran tunggal yang menyeimbangkan kedua matriks tersebut.

Pemilihan matriks evaluasi yang paling relevan sangat bergantung pada tujuan aplikasi model dan biaya relatif yang terkait dengan setiap jenis kesalahan klasifikasi. Dalam praktiknya, analisis kinerja model yang komprehensif sering kali melibatkan peninjauan beberapa metrik secara bersamaan untuk mendapatkan pemahaman yang holistik (Powers, 2011; Tharwat, 2021). Penggunaan metrik Akurasi, Presisi, *Recall*, dan *F1-Score*, seperti yang diadopsi dalam referensi utama penelitian ini (Rohman et al., 2025), merupakan praktik umum dalam evaluasi model klasifikasi.

2.13. Validasi Model *Machine Learning*

Validasi model merupakan tahapan penting dalam penelitian *machine learning*. Hal ini bertujuan untuk menilai kemampuan model dalam melakukan generalisasi terhadap data baru yang belum pernah dilihat oleh model sebelumnya. Proses validasi dilakukan untuk memastikan bahwa model yang dibangun tidak hanya memiliki kinerja yang baik pada data latih, tetapi mampu memberi prediksi yang andal ketika diterapkan pada data uji. Validasi model membantu mengurangi risiko terjadinya *overfitting*.

Dalam penelitian ini, validasi model dilakukan menggunakan pendekatan *hold-out validation*, yaitu dengan membagi *dataset* ke dalam dua bagian, yaitu data latih dan data uji. Skema pembagian data ini merupakan salah satu metode validasi yang umum digunakan dalam penelitian *machine learning* karena bersifat sederhana, efisien, dan mudah untuk diimplementasikan. Pendekatan ini juga sesuai

untuk penelitian yang berfokus pada perbandingan performa beberapa model pada satu skema data yang sama.

Evaluasi performa model pada data uji dilakukan menggunakan metrik evaluasi klasifikasi yang relevan seperti *accuracy*, *precision*, *recall*, dan *F1-Score*. Metrik-metrik tersebut digunakan untuk mengukur kinerja model dari berbagai aspek seperti ketepatan prediksi secara keseluruhan serta kemampuan model dalam mengidentifikasi setiap kelas secara tepat. Hasil evaluasi inilah yang kemudian digunakan sebagai dasar dalam membandingkan performa model tunggal dan model *hybrid* yang dikembangkan dalam penelitian ini. Dengan menerapkan teknik validasi yang tepat, peneliti dapat membangun kepercayaan terhadap hasil evaluasi model dan memastikan bahwa model yang dihasilkan tidak hanya berkinerja baik pada data yang telah dilihat tetapi juga siap untuk generalisasi pada aplikasi dunia nyata (Refaeilzadeh et al., 2009; James et al., 2013).

BAB III

METODE PENELITIAN

3.1. Alat Penelitian

Spesifikasi dari perangkat lunak dan perangkat keras yang akan digunakan peneliti dalam melakukan penelitian ini adalah sebagai berikut:

Perangkat Lunak:

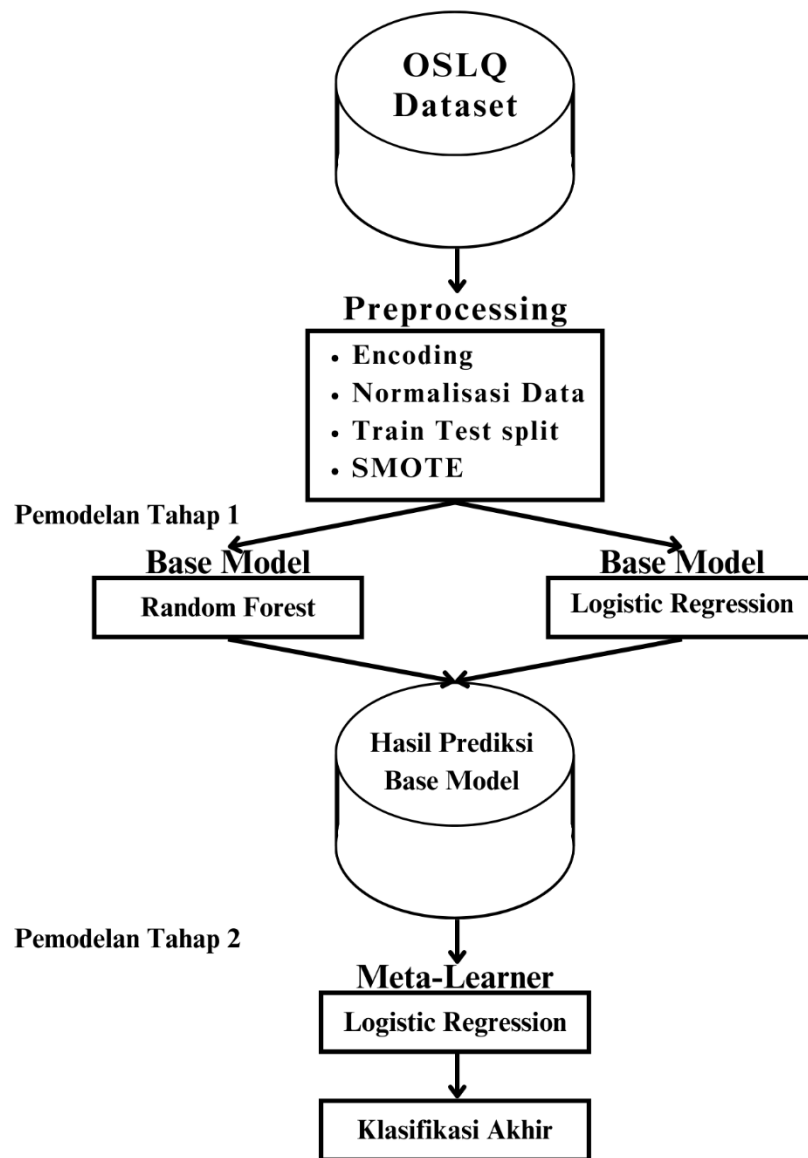
- Sistem Operasi : Windows 11 64-bit
- Alat Perekapan Data : Google Spreadsheet, Google Colaboratory
- Bahasa Pemrograman : Python

Perangkat Keras

- Processor : AMD Ryzen 3 3200 G
- Memory : 16 GB

3.2. Tahapan Penelitian

Penelitian ini dilakukan melalui beberapa tahapan. Tahapan-tahapan tersebut dapat dilihat pada gambar 3.1.



Gambar 3. 1. Alur Penelitian Hybrid Machine Learning

3.3. Pengumpulan dan Penyusunan *Dataset*

Dataset yang akan digunakan penelitian ini merupakan data penelitian yang dikumpulkan oleh Bapak Dr. Irwanto, M.Pd., dari mahasiswa Universitas Negeri Jakarta. *Dataset* ini berisi hasil survei mengenai strategi belajar mandiri mahasiswa dalam konteks pembelajaran daring. Instrumen utama yang digunakan untuk

pengumpulan data asli adalah *Online Self-Regulated Learning* (OSLQ), yang dikembangkan oleh Bernard et al. (2019) dan telah diadaptasikan ke dalam Bahasa Indonesia. *Dataset* yang diperoleh mencakup respons sebanyak 577 mahasiswa dan terdiri dari 39 variabel.

Variabel - variabel dalam *dataset* ini dapat dikategorikan menjadi dua kelompok utama yaitu, informasi demografi responden yang meliputi atribut seperti fakultas, jenis kelamin, tahun akademik, kelompok usia, jumlah saudara, lokasi tempat tinggal, sumber biaya kuliah, penggunaan internet harian, tujuan penggunaan internet, perangkat yang sering digunakan, serta latar belakang pendidikan dan pekerjaan orang tua. kategori kedua yaitu respons kuesioner mahasiswa yang terdiri dari 24 pertanyaan yang mengukur enam dimensi strategi belajar mandiri secara daring di antaranya penetapan tujuan, penataan lingkungan, strategi tugas, manajemen waktu, mencari bantuan, dan evaluasi diri. respons diberikan dalam skala likert 5 poin (sangat setuju, setuju, ragu-ragu, tidak setuju, sangat tidak setuju) yang dikodekan secara numerik. kategori terakhir yaitu IPK mahasiswa yang telah dikategorikan ke dalam 5 kelompok. variabel IPK ini akan berfungsi sebagai variabel target yang akan diprediksi dalam penelitian ini.

Berikut bagian Informasi Demografi Responden:

Tabel 3. 1. Demografi Mahasiswa

No	Demografi	Jawaban
1	Fakultas	Fakultas Ekonomi
		Fakultas Ilmu Pendidikan
		Fakultas Pendidikan Psikologi
		Fakultas Teknik
		Fakultas Bahasa dan Seni

No	Demografi	Jawaban
		Fakultas Matematika dan IPA
		Fakultas Ilmu Sosial
		Fakultas Ilmu Olahraga
2	Jenis Kelamin	Perempuan
		Laki-laki
3	Tahun Akademik	Tahun pertama
		Tahun kedua
		Tahun ketiga
		Tahun keempat
4	Usia	18-20 tahun
		21-23 tahun
		Lebih dari 23 tahun
5	Waktu belajar sebelum Covid-19	Kurang dari 4 jam
		4 hingga 7 jam
		Lebih dari 7 jam
6	Waktu belajar selama Covid-19	Kurang dari 4 jam
		4 hingga 7 jam
		Lebih dari 7 jam
7	Jumlah saudara	Satu
		Dua
		Tiga
		Empat atau lebih
8	Lokasi tempat tinggal	Pedesaan

No	Demografi	Jawaban
		Pinggiran Kota
		Perkotaan
9	Sumber biaya kuliah	Orang tua
		Beasiswa
		Bekerja untuk membiayai diri sendiri
10	Indeks Prestasi Kumulatif (IPK)	Di atas 3,5
		3,0-3,49
		2,5-2,99
		2,0-2,49
		Di bawah 2,0
11	Penggunaan internet harian	Kurang dari 1 jam
		1-6 jam per hari
		12 jam per hari
12	Tujuan penggunaan internet	Akademik
		Non-akademik
		Kedua-duanya
13	Perangkat yang sering digunakan	Komputer/PC
		Laptop
		<i>Smartphone</i>
		Tablet atau <i>Ipad</i>
14	Pendidikan Terakhir Orang Tua	Sekolah Dasar (SD)
		Sekolah Menengah (SMP, SMA)
		Diploma (D1, D2, D3)

No	Demografi	Jawaban
		Sarjana (S1)
		Pascasarjana (S2, S3)
15	Pekerjaan orang tua	Tidak bekerja
		Wiraswasta
		Buruh
		Karyawan Swasta
		Pegawai Pemerintah

Berikut bagian kuesioner regulasi diri dalam belajar:

Tabel 3. 2. Kuesioner Regulasi Diri dalam Belajar

Kode Pertanyaan	Pernyataan	Tujuan Pernyataan
A. Penetapan Tujuan (Goal Setting - GS)		
GS1	Saya menetapkan standar untuk tugas-tugas saya selama pembelajaran daring	Menilai apakah mahasiswa menetapkan standar kualitas untuk tugas-tugas daring mereka.
GS2	Saya memiliki target jangka pendek (harian atau mingguan) dan jangka panjang (bulanan atau semesteran)	Mengukur apakah mahasiswa membuat target belajar yang spesifik berdasarkan waktu (jangka pendek dan panjang).
GS3	Saya menetapkan standar yang tinggi selama menjalani pembelajaran daring	Menilai apakah mahasiswa memiliki ekspektasi standar yang tinggi terhadap diri sendiri dalam pembelajaran daring.
GS4	Saya menetapkan target untuk memudahkan saya mengatur waktu belajar selama pembelajaran daring	Mengukur apakah penetapan target digunakan sebagai alat untuk membantu manajemen waktu belajar daring.

Kode Pertanyaan	Pernyataan	Tujuan Pernyataan
GS5	Saya tetap menjaga kualitas pekerjaan saya meskipun pembelajaran dilaksanakan secara daring	Menilai komitmen mahasiswa untuk menjaga kualitas hasil kerja dalam konteks daring.
B. Penataan Lingkungan (<i>Environment Structuring</i> - ES)		
ES1	Saya memilih lokasi untuk belajar daring yang dapat terhindar dari berbagai gangguan	Menilai upaya mahasiswa mencari lingkungan belajar daring yang minim distraksi.
ES2	Saya mencari tempat yang nyaman untuk belajar	Mengukur apakah mahasiswa secara aktif mencari kenyamanan fisik dalam lingkungan belajarnya.
ES3	Saya tahu di mana saya dapat belajar paling efisien selama pembelajaran daring	Menilai kesadaran mahasiswa akan tempat atau kondisi yang paling mendukung efisiensi belajarnya secara daring.
ES4	Saya memilih waktu dengan sedikit gangguan untuk belajar selama mengikuti kelas daring	Mengukur strategi mahasiswa dalam memilih waktu belajar yang minim interupsi untuk pembelajaran daring.
C. Strategi Tugas (<i>Task Strategies</i> - TS)		
TS1	Saya berusaha membuat catatan yang lebih detail karena catatan tersebut penting selama belajar di kelas daring dibandingkan belajar di kelas tradisional	Menilai upaya mahasiswa dalam membuat catatan yang komprehensif sebagai strategi belajar daring.

Kode Pertanyaan	Pernyataan	Tujuan Pernyataan
TS2	Saya membaca dengan keras setiap materi perkuliahan yang tertera pada kelas daring untuk menghindari gangguan	Mengukur penggunaan strategi membaca dengan suara keras untuk meningkatkan fokus dan pemahaman materi daring.
TS3	Saya terbiasa menyiapkan pertanyaan sebelum bergabung dalam ruang obrolan dan diskusi daring	Menilai keaktifan mahasiswa dalam mempersiapkan diri untuk interaksi dan diskusi daring.
TS4	Untuk dapat menguasai materi, saya mengerjakan tugas tambahan lebih banyak selain yang diberikan pada kelas daring	Mengukur inisiatif mahasiswa untuk memperdalam pemahaman materi melalui pengerjaan tugas tambahan.
D. Manajemen Waktu (<i>Time Management</i> - TM)		
TM1	Saya mengalokasikan waktu belajar lebih banyak karena saya sadar bahwa kelas daring membutuhkan lebih banyak waktu	Menilai kesadaran mahasiswa akan kebutuhan waktu ekstra untuk belajar daring dan tindakan alokasinya.
TM2	Saya mencoba menjadwalkan satu waktu tertentu untuk belajar setiap hari atau setiap minggu, kemudian saya mengamati jadwal tersebut	Mengukur kemampuan mahasiswa dalam membuat dan mematuhi jadwal belajar yang teratur.

Kode Pertanyaan	Pernyataan	Tujuan Pernyataan
TM3	Meskipun tidak dapat hadir di kelas setiap hari, saya tetap berusaha membagi waktu belajar secara merata setiap harinya	Menilai konsistensi mahasiswa dalam mendistribusikan waktu belajar harian untuk pembelajaran daring.
E. Mencari Bantuan (<i>Help Seeking</i> - HS)		
HS1	Saya mencari seseorang yang memahami materi dalam pembelajaran daring agar saya dapat berkonsultasi dengannya ketika saya membutuhkan bantuan	Menilai proaktivitas mahasiswa dalam mengidentifikasi dan mencari sumber bantuan ahli.
HS2	Saya membagikan masalah saya dengan teman sekelas saya agar kami tahu kesulitan yang kami hadapi dan bagaimana cara menyelesaikan kesulitan tersebut	Mengukur kecenderungan mahasiswa untuk berkolaborasi dengan teman sebaya dalam mengatasi kesulitan belajar.
HS3	Jika diperlukan, saya mencoba untuk bertemu langsung dengan teman sekelas saya	Menilai upaya mahasiswa untuk mencari interaksi tatap muka dengan teman sekelas sebagai bentuk dukungan.
HS4	Saya gigih mencari bantuan dari dosen melalui email atau <i>WhatsApp</i>	Menilai kegigihan mahasiswa dalam mencari bantuan dari dosen menggunakan berbagai media komunikasi.
F. Evaluasi Diri (<i>Self-Evaluation</i> - SE)		

Kode Pertanyaan	Pernyataan	Tujuan Pernyataan
SE1	Saya merangkum apa yang telah saya pelajari dari kelas daring untuk menilai pemahaman saya tentang apa yang telah saya pelajari	Menilai penggunaan strategi merangkum sebagai cara untuk mengevaluasi pemahaman diri.
SE2	Saya sering bertanya pada diri sendiri tentang materi perkuliahan ketika belajar di kelas daring	Mengukur praktik refleksi diri mahasiswa terkait pemahaman materi perkuliahan daring.
SE3	Saya berkomunikasi dengan teman sekelas untuk mengetahui apa yang harus saya lakukan di kelas daring	Menilai upaya mahasiswa mencari panduan dan klarifikasi dari teman sekelas mengenai ekspektasi tugas.
SE4	Saya berkomunikasi dengan teman sekelas untuk mengetahui apakah yang saya pelajari berbeda dengan apa yang mereka pelajari	Mengukur penggunaan diskusi dengan teman sebaya sebagai sarana untuk membandingkan dan memvalidasi pemahaman.

3.4. Tahapan *Pre-Processing Data*

Sebelum melakukan pemodelan *hybrid machine learning*, *dataset* yang telah disusun sebelumnya akan diolah terlebih dahulu dan dilakukan pra-pemrosesan data. Tahapan ini melalui proses sebagai berikut:

1. *Dataset* dianalisis terlebih dahulu untuk memastikan tidak ada *missing values* atau duplikasi data. Jika ditemukan, maka *missing values* ditangani dengan penghapusan.

2. *Outlier* dideteksi pada variabel numerik seperti *Learning Time*, *Daily Internet Usage*, dan skor dimensi strategi belajar daring menggunakan metode IQR. Nilai yang berada di luar batas bawah dan atas dianggap sebagai *outlier* dan dihapus untuk menjaga efisiensi serta stabilitas model.
3. Data kategorik seperti fakultas, jenis kelamin, atau lokasi tempat tinggal diubah menjadi bentuk numerik menggunakan *one-hot encoding*. Hal ini dilakukan agar data dapat diproses oleh algoritma *machine learning*.
4. Variabel numerik dinormalisasi menggunakan metode *z-score standardization* sehingga setiap fitur memiliki rata-rata 0 dan standar deviasi 1. Normalisasi dilakukan karena data akan diproses dengan algoritma Regresi Logistik yang sensitif terhadap skala data.
5. *Dataset* dibagi menjadi dua bagian, yaitu data latih sebanyak 80% dan data uji sebanyak 20% dengan metode *stratified split* agar distribusi kelas pada data latih dan data uji tetap seimbang.
6. Karena distribusi variabel tidak seimbang maka perlu dilakukan proses penyeimbang data pada data latih menggunakan metode *Synthetic Minority Oversampling Method (SMOTE)*.

Setelah semua tahapan pra pemrosesan data selesai, maka akan didapatkan *dataset* yang bersih yang sudah siap untuk masuk ke dalam pemodelan *machine learning*.

3.5. Tahapan Pemodelan *Hybrid Machine Learning*

Setelah tahapan *pre-processing data* selesai, tahapan selanjutnya adalah pemodelan menggunakan pendekatan *hybrid machine learning*. Model *hybrid* yang digunakan dalam penelitian ini merupakan bentuk khusus dari *ensemble learning*, yaitu metode *stacking ensemble*. Penelitian ini akan menggunakan model *hybrid* dengan dua algoritma, yaitu *Random Forest* sebagai *base model* pertama dan *Logistic Regression* sebagai *base model* kedua sekaligus *meta-learner* pada tahap akhir.

Tahapan pertama adalah membangun model *Random Forest* sebagai *base model* pertama. Setiap *Decision tree* di dalam *Random Forest* dibentuk melalui langkah-langkah berikut:

1. Pemilihan subset fitur secara acak.
2. Pemisahan *node* yang dilakukan berdasarkan nilai *Gini Impurity* untuk menentukan titik *split* optimal.
3. Proses pemisahan berlanjut secara rekursif hingga kondisi berhenti terpenuhi.
4. Setiap *leaf node* diberi label kelas berdasarkan hasil *majority voting* dari sampel data pada *node* tersebut.
5. Setelah seluruh *Decision tree* terbentuk, *Random Forest* dibangun dengan cara menggabungkan semua *Decision tree* menggunakan metode *Bootstrap aggregating* di mana setiap pohon dilatih pada subset data latih yang diambil secara acak dengan metode *sampling with replacement* dari *dataset*.
6. Prediksi akhir *Random Forest* diperoleh melalui proses *majority voting* dari semua *Decision tree* yang terbentuk.

Tahapan kedua adalah membangun model *Logistic Regression* sebagai *base model* kedua. Model ini bekerja dengan menghitung probabilitas suatu kelas melalui fungsi logit dengan parameter yang diestimasi menggunakan metode *Maximum Likelihood Estimation* (MLE). Model ini menghasilkan output probabilitas untuk setiap kelas IPK berdasarkan kombinasi variabel prediktor *dataset*.

Setelah kedua *base model* selesai dilatih, masing-masing model menghasilkan prediksi probabilitas kelas. Nilai probabilitas yang dihasilkan kedua model kemudian disimpan sebagai *meta-features*, yang akan digunakan sebagai *input* bagi *meta-learner*. Model *meta-learner* yang akan digunakan adalah *Logistic Regression*. Model ini dilatih untuk mempelajari hubungan antara hasil prediksi dari dua *base model* dan menentukan kombinasi yang menghasilkan performa klasifikasi terbaik. Proses prediksi pada data uji dilakukan dengan urutan sebagai berikut:

1. Hasil prediksi probabilitas kedua model digunakan sebagai *input meta-learner* yang berupa model *Logistic Regression*.
2. *Meta-learner* menghasilkan prediksi akhir berupa kategori IPK mahasiswa.

Setelah model *hybrid* selesai dilatih, dilakukan proses evaluasi kinerja model untuk menilai kemampuan klasifikasi tiap model. Evaluasi dilakukan terhadap model tunggal (RF dan LR) dan model *hybrid* menggunakan beberapa metrik seperti *confusion matrix*, *accuracy*, *precision*, *recall*, dan *F1 score*. Metrik-metrik tersebut digunakan untuk menilai keseimbangan antara kemampuan model dalam mengklasifikasi data, serta untuk memastikan peningkatan performa model *hybrid* dibandingkan model tunggal.

BAB IV

HASIL DAN PEMBAHASAN

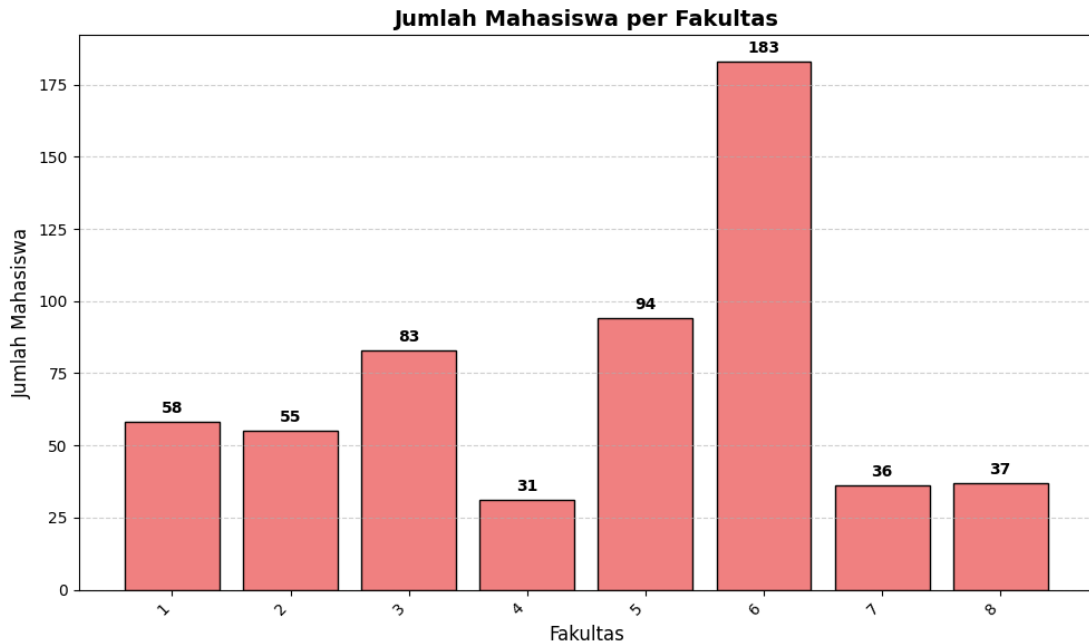
4.1. Data Exploration

Berdasarkan hasil eksplorasi data, diketahui *dataset* memiliki 577 observasi lengkap tanpa adanya nilai kosong. Nilai IPK mahasiswa memiliki 5 kategori yaitu kategori satu (1) yang mewakili nilai IPK di atas 3,5, kategori dua (2) yang mewakili nilai IPK 3,0 sampai 3,49, kategori tiga (3) yang mewakili nilai IPK 2,5 sampai 2,99, kategori empat (4) yang mewakili nilai IPK 2,0 sampai 2,49, dan kategori 5 yang mewakili nilai IPK di bawah 2,0.

Tabel 4. 1. Kelas IPK

<i>Code</i>	<i>Cumulative average (GPA)</i>	<i>count</i>
1	3.5+	410
2	3.0 – 3.49	134
3	2.5 – 2.99	26
4	2.0 – 2.49	4
5	<2.0	3

Tahapan eksplorasi data selanjutnya dilakukan analisis deskriptif terhadap distribusi mahasiswa dari beberapa fakultas yang terdiri dari Fakultas Ekonomi (1), Fakultas Pendidikan (2), Fakultas Pendidikan Psikologi (3), Fakultas Teknik (4), Fakultas Bahasa dan Seni (5), Fakultas Matematika dan Ilmu Pengetahuan Alam (6), Fakultas Ilmu Sosial (7), dan Fakultas Ilmu Olahraga (8). Distribusi responden dapat dilihat pada gambar 4.1.:



Gambar 4. 1. Diagram Jumlah Mahasiswa per Fakultas

4.2.Data Pre-processing

Outlier merupakan nilai ekstrem yang memiliki perbedaan signifikan dibandingkan sebagian besar data lainnya. Keberadaan *outlier* dapat memengaruhi distribusi data dan menurunkan kinerja model, terutama pada algoritma yang sensitif terhadap distribusi data. Pada penelitian ini, deteksi *outlier* dilakukan menggunakan metode *Interquartile Range* (IQR). Deteksi *outlier* difokuskan pada fitur numerik yang berasal dari hasil agregasi IPK. Penghapusan *outlier* dilakukan pada fitur numerik kontinu seperti *Daily Internet Usage*, *Learning Time Before COVID-19*, *Learning Time During COVID-19*, dan *No. of Siblings*. Akan tetapi, siswa yang berprestasi rendah tidak boleh dianggap sebagai gangguan, karena mereka mewakili

kasus-kasus penting untuk dukungan pengambilan keputusan dan intervensi (Romero & Ventura, 2010). Literatur menyatakan bahwa nilai ekstrem yang merepresentasikan individu nyata tidak seharusnya dihapus tanpa indikasi kesalahan pengukuran (Tabachnick et al. 2013; Field, 2018). Oleh karena itu, *outlier* tidak dihapus dan penanganan dilakukan melalui normalisasi data sebagaimana disarankan dalam *pre-processing machine learning* untuk menjaga informasi penting dalam data (Han et al, 2012).

Selanjutnya dilakukan penggabungan kelas IPK <2,0, IPK 2,0-2,49, dan IPK 2,5-2,99 menjadi satu kelas IPK. Penggabungan ini dilakukan karena jumlah sampel data pada masing-masing kelas tersebut relatif sedikit sehingga berpotensi menimbulkan masalah ketidakseimbangan kelas yang dapat menurunkan kinerja dan stabilitas model klasifikasi. Dalam konteks *machine learning*, kelas dengan jumlah sampel yang kecil sering kali sulit dipelajari oleh model dan dapat menyebabkan bias prediksi terhadap kelas mayoritas. He dan Gracia (2009) menyatakan bahwa ketidakseimbangan kelas yang ekstrem dapat menghambat kemampuan model dalam mempelajari pola kelas minoritas secara efektif. Selain itu, Geron (2019) menjelaskan bahwa penggabungan kelas yang memiliki karakteristik serupa merupakan pendekatan yang wajar apabila perbedaan antar kelas tidak signifikan secara substantif dan bertujuan untuk meningkatkan reliabilitas model. Secara konseptual, ketiga rentang IPK tersebut sama-sama merepresentasikan kelompok mahasiswa dengan prestasi akademik rendah, sehingga penggabungan kelas tidak menghilangkan makna akademik dari variabel target, tetapi justru membantu model dalam mempelajari pola yang lebih stabil dan representatif.

Pada *dataset* yang akan digunakan, terdapat beberapa variabel dengan tipe kategorikal. Agar data tersebut dapat diolah oleh algoritma *machine learning*, variabel ini perlu diubah menjadi bentuk numerik melalui proses *encoding*. Variabel kategorikal dibagi menjadi dua jenis, yang pertama variabel nominal yang tidak memiliki urutan seperti kolom *faculty*, *gender*, *Location of residence*, *source of tuition fees*, *Device often use*, dan *Parental occupation*. Jenis variabel kedua yaitu

variabel ordinal di mana tiap angka memiliki urutan logis seperti kolom *Age group*, *Academic year*, *Cumulative average (GPA)*, *Daily internet usage*, *Learning time before COVID-19*, *Learning time during COVID-19*, *Parental education*, dan *No. of siblings*. Tahapan *encoding* dimulai dengan memisahkan kolom ordinal dan nominal. *One-Hot Encoding* diterapkan pada kolom nominal, sedangkan untuk kolom ordinal diterapkan *Ordinal Encoding*.

```
ordinal_cols = [  
    'Age group', 'Academic year', 'Cumulative average (GPA)',  
    'Daily Internet usage', 'Learning time before COVID-19',  
    'Learning time during COVID-19', 'Parental education', 'No. of Siblings'  
]  
  
nominal_cols = [  
    'Faculty', 'Gender', 'Location of residence',  
    'Source of tuition fees ', 'Devices often use', 'Parental occupation'  
]
```

Gambar 4. 2. Kolom Ordinal dan Nominal

Setelah mengkonversi variabel kategorikal menjadi bentuk numerik melalui proses *encoding*, tahap selanjutnya adalah melakukan normalisasi (*scaling*) pada data OSLQ. Proses ini dilakukan untuk menyetarakan skala antar fitur sehingga setiap variabel memiliki kontribusi yang seimbang dalam proses pembentukan model. Tahapan ini dimulai dengan memilih kolom yang termasuk dalam variabel OSLQ diantaranya *Goal Setting* (GS1-GS5), *Environment Structuring* (ES1-ES4), *Task Strategies* (TS1-TS4), *Time Management* (TM1-TM3), *Help Seeking* (HS1-HS4), dan *Self-Evaluation* (SE1-SE4). Proses normalisasi dilakukan menggunakan *min max scaler*.

Setelah seluruh variabel dalam *dataset* dikonversi ke bentuk ordinal dan numerik serta telah dilakukan normalisasi pada kolom OSLQ, tahapan berikutnya adalah memisahkan variabel prediktor (X) dan variabel target (y). Variabel prediktor (X) berisi semua fitur independen yang akan digunakan model untuk mempelajari hubungan antar variabel. Variabel target (y) berisi nilai yang akan diprediksi model, yaitu kategori IPK mahasiswa (*Cumulative average (GPA)*).

Setelah memisahkan variabel prediktor (X) dan variabel target (y), selanjutnya adalah membagi *dataset* menjadi dua *subset*, yaitu data latih (*data train*)

yang akan digunakan untuk melatih model agar mengenali pola pada data, dan data uji (*data test*) yang akan digunakan untuk mengevaluasi performa mode terhadap data baru yang belum pernah dilihat oleh model sebelumnya. pada penelitian ini, digunakan pembagian 80% untuk data latih dan 20% data untuk pengujian, dengan metode *stratified sampling* untuk menjaga porsi kelas IPK di kedua subset tetap seimbang. Hasil pembagian pada data dapat dilihat pada gambar berikut:

```
Jumlah data latih : 461
Jumlah data uji   : 116
Proporsi kelas di data latih:
  IPK_final
3.5+      0.711497
3.0-3.49  0.232104
<3.0      0.056399
Name: proportion, dtype: float64

Proporsi kelas di data uji:
  IPK_final
3.5+      0.706897
3.0-3.49  0.232759
<3.0      0.060345
Name: proportion, dtype: float64
```

Gambar 4. 3. Proporsi Kelas Data Latih dan Uji

Setelah dilakukan pembagian data latih dan data uji, hasil analisis distribusi kelas pada variabel target menunjukkan adanya ketidakseimbangan kelas. Kelas IPK diatas 3,5 mendominasi *dataset*, sementara kelas dengan IPK <3,0 memiliki jumlah data yang jauh lebih sedikit. Maka perlu dilakukan metode *oversampling* menggunakan teknik SMOTE. Teknik SMOTE bekerja dengan menambah jumlah data pada kelas minoritas dengan membuat sampel sintetik baru berdasarkan jarak *k-nearest neighbors*. Berikut adalah distribusi pada data latih sebelum dan sesudah SMOTE:

```

Distribusi sebelum SMOTE (kategori):
IPK_final
3.5+      328
3.0-3.49  107
<3.0      26
Name: count, dtype: int64

Distribusi sesudah SMOTE (kategori):
IPK_final
3.0-3.49  328
3.5+      328
<3.0      328
Name: count, dtype: int64

```

4.3. Modeling

Setelah melalui tahapan *pre-processing*, data yang telah bersih dan diubah ke dalam bentuk numerik kemudian, proses selanjutnya adalah tahapan pembangunan model (*modeling*). Pada penelitian ini, pendekatan yang digunakan adalah *Hybrid Machine Learning* dengan skema *Stacking Ensemble*. Berbeda dengan penggunaan model tunggal, metode ini mengintegrasikan prediksi dari beberapa algoritma pembelajaran yang berbeda untuk membentuk satu keputusan akhir yang lebih kuat. Proses pemodelan ini dilakukan secara bertingkat yang diawali dengan *base model* dan *meta learner*. Seluruh pembuatan model pada penelitian ini baik *base model* maupun *meta learner* dibangun secara manual menggunakan komputasi matriks untuk memahami proses pembobotan fitur secara mendalam.

4.3.1. Pembangunan Base Model 1: *Random Forest*

Model dasar pertama yaitu *Random Forest* yang diimplementasikan dengan metode *Bagging*. Algoritma ini membangun kumpulan *Decision Trees* dari sub-sampel data yang diambil secara acak dengan pengembalian. Setiap *Decision Trees* dilatih untuk memisahkan fitur perilaku belajar mahasiswa untuk memprediksi kategori IPK. Karena model ini terdiri dari banyak pohon, keputusan akhir diambil berdasarkan mekanisme *voting*. Penggunaan *Random Forest* pada lapisan dasar untuk menangani kompleksitas fitur non-linear dan mengurangi varians yang terjadi pada pohon tunggal.

Untuk mencapai performa model yang optimal, dilakukan pengaturan parameter pada model. Berikut adalah konfigurasi parameter yang digunakan dalam penelitian ini:

Tabel 4. 2. Parameter *Random Forest*

Parameter	Konfigurasi	Keterangan
n_trees	128	Jumlah pohon dalam RF
max_depth	10	Kedalaman maksimum setiap pohon
min_size	5	Jumlah sampel minimum tiap <i>node</i>
n_features	sqrt	Jumlah fitur yang dipilih acak saat split
sample_size	1.0	Rasio sampel data untuk <i>bootstrapping</i>

4.3.2. Pembangunan Base Model 2: *Logistic Regression*

Model dasar kedua adalah *Logistic Regression*, mengingat variabel target (IPK) memiliki lebih dari dua kelas (multi-kelas), penelitian ini menerapkan strategi OvR. Metode ini, algoritma melatih satu model biner terpisah untuk setiap kategori IPK. Proses pelatihan model ini menggunakan optimasi *Gradient Descent* untuk meminimalkan *loss function* secara iteratif. Fungsi aktivasi *Sigmoid* digunakan untuk memetakan hasil perhitungan linear ke dalam probabilitas antara 0 hingga 1. Adapun pengaturan parameter untuk mencapai hasil yang optimal adalah sebagai berikut:

Tabel 4. 3. Parameter *Logistic Regression*

Parameter	Konfigurasi	Keterangan
Learning Rate	0.001	Laju pembelajaran bobot
Epoch	2000	Jumlah iterasi pelatihan maksimum
Early Stopping	Patience: 100	Penghentian otomatis jika <i>loss</i> stagnan

4.3.3. Pembangunan *Meta Learner*

Komponen akhir pada model *hybrid* ini adalah *Meta Learner* yang berada di lapisan akhir. Algoritma yang digunakan sebagai *Meta Learner* adalah *Logistic Regression*. Tugas utama model ini adalah mempelajari pola kesalahan (*error pattern*) dan korelasi antara prediksi yang dihasilkan oleh kedua *base model* yang telah dibangun sebelumnya. Proses ini melibatkan pembentukan *dataset* baru yaitu *Meta Features*, di mana hasil prediksi probabilitas dari *Random Forest* dan *Logistic Regression* digabungkan menjadi fitur input baru (x_{new}). *Meta Learner* menggunakan hasil prediksi *base model* yang sudah dikumpulkan di *Meta Features* sebagai input dan memberi bobot yang optimal kepada model yang performanya lebih baik pada sampel data tertentu, sehingga akurasi prediksi akhir dapat ditingkatkan dibandingkan model tunggal. *Meta Learner* menggunakan konfigurasi parameter yang sama dengan konfigurasi *base model Logistic Regression*.

4.4. Hasil Penelitian

Setelah model berhasil dibangun, tahap selanjutnya adalah melakukan pengujian untuk mengukur kinerja model dalam memprediksi prestasi akademik mahasiswa. Pengujian dilakukan menggunakan data uji sebanyak 107 sampel yang tidak dilibatkan dalam proses pelatihan maupun proses penanganan ketidakseimbangan data. Hal ini bertujuan untuk memastikan objektivitas hasil evaluasi pada data riil. Evaluasi kinerja dilakukan terhadap tiga skenario model, yaitu model tunggal *Random Forest*, model tunggal *Logistic Regression*, dan model *Hybrid Stacking*. Metrik evaluasi yang digunakan meliputi *Accuracy* untuk mengukur ketepatan global, *precision*, *recall*, dan *F1-Score* untuk menganalisa detail kinerja pada setiap kelas IPK.

4.4.1. Hasil Pengujian *Base Model 1: Random Forest*

Pengujian pertama dilakukan menggunakan model *Random Forest*. Berdasarkan hasil evaluasi pada data uji, model ini menghasilkan akurasi sebesar 71%. Rincian performa model untuk setiap kelas IPK ditunjukkan pada tabel 4.4

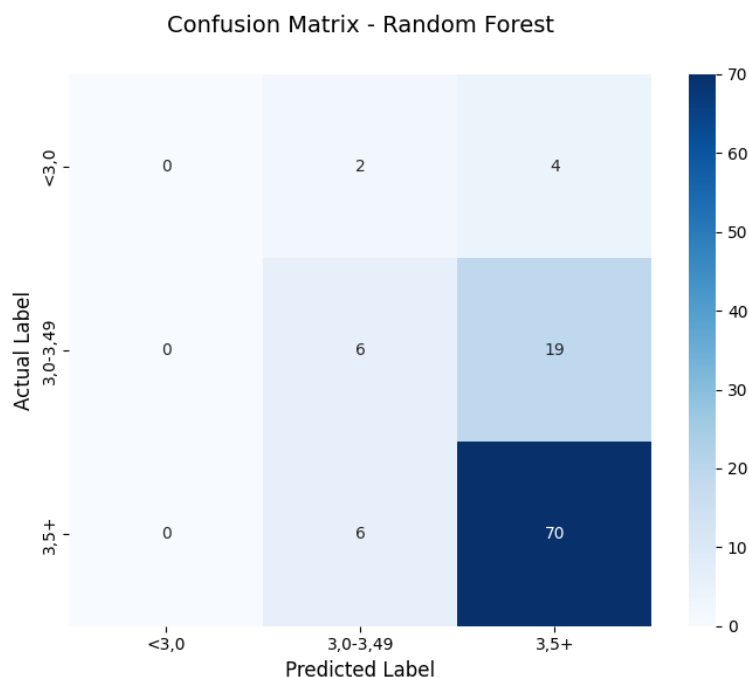
melalui nilai *precision*, *recall*, dan *F1-Score*, serta divisualisasikan dalam *confusion matrix*:

Tabel 4. 4. *Classification Report Random Forest*

Kelas IPK	TP	FP	FN	<i>Precision</i>	<i>Recall</i>	<i>F1-Score</i>	<i>Support</i>
3.5+	70	23	6	0,75	0,92	0,83	76
3.0-3.49	6	8	19	0,43	0,24	0,31	25
<3,0	0	0	6	0,00	0,00	0,00	6
<i>Accuracy</i>						0,71	107
<i>Macro avg</i>				0,39	0,39	0,38	107
<i>Weighted Avg</i>				0,63	0,71	0,66	107

Berdasarkan *classification report* pada tabel 4.4, model *Random Forest* menunjukkan kinerja yang sangat baik pada kelas IPK 3,5+ sebagai kelas mayoritas. Dari total 76 mahasiswa pada kelas tersebut, sebanyak 70 mahasiswa berhasil diprediksi dengan benar (TP), sementara 6 mahasiswa salah diklasifikasikan ke kelas IPK 3,0-3,49 (FN). Nilai *recall* yang tinggi pada kelas IPK 3,5+ (0,92) menunjukkan bahwa model sangat efektif dalam mengenali mahasiswa dengan IPK tinggi. Akan tetapi, performa model menurun secara signifikan pada kelas IPK 3,0-3,49 dan IPK <3,0. Pada kelas IPK 3,0-3,49, hanya 6 mahasiswa dari 25 mahasiswa yang berhasil diklasifikasikan dengan benar (TP), sementara 19 mahasiswa salah diklasifikasikan sebagai IPK 3,5+ (FN). Kondisi ini menyebabkan nilai *recall* pada kelas IPK menengah menjadi rendah (0,24), yang menandakan bahwa sebagian besar mahasiswa pada kelas ini tidak berhasil dikenali oleh model. Selanjutnya pada kelas IPK <3,0, model *Random Forest* tidak berhasil mengklasifikasikan satu pun mahasiswa dengan benar (TP). Seluruh mahasiswa pada kelas ini salah diklasifikasikan ke kelas yang lebih

tinggi, sehingga nilai *precision*, *recall*, dan *F1-Score* untuk kelas IPK <3,0 bernilai 0,00. Hal ini menunjukkan bahwa model sepenuhnya gagal mengenali pola pada kelas minoritas dengan jumlah data yang sangat terbatas. Perbedaan yang cukup besar antara nilai *accuracy* (0,71) dan *Macro Average F1-Score* (0,38) mengindikasikan adanya bias model terhadap kelas mayoritas. Nilai *accuracy* yang relatif tinggi terutama dipengaruhi oleh keberhasilan model dalam memprediksi kelas IPK 3,5+, sedangkan *Macro Average F1-Score* memberi bobot yang sama pada seluruh kelas dan secara jelas mencerminkan performa buruk pada kelas minoritas.



Gambar 4. 4. Confusion Matrix Random Forest

Berdasarkan analisis pada gambar 4.4 dapat disimpulkan bahwa model *Random Forest* memiliki bias kuat terhadap kelas mayoritas. Model cenderung meningkatkan jumlah TP untuk kelas 3,5+ dengan mengorbankan FN yang tinggi di kelas IPK menengah dan rendah. Menurut He dan Gracia (2009), kondisi seperti ini umum terjadi pada *dataset* pendidikan dengan distribusi kelas tidak seimbang, di mana model cenderung mengabaikan kelas minoritas karena

kontribusinya terhadap *total error* sangat kecil. Oleh karena itu, strategi penyeimbang data seperti *oversampling* atau *class-weight adjustment* perlu dipertimbangkan untuk meningkatkan sensitivitas model terhadap kelas minoritas.

Feature Importance (Random Forest) – 10 Teratas:	
GS5	: 0.0334
TM1	: 0.0333
Academic year	: 0.0331
GS3	: 0.0314
TS4	: 0.0305
TS3	: 0.0303
TS1	: 0.0300
TS2	: 0.0294
HS1	: 0.0276
SE3	: 0.0274

Gambar 4. 5. Feature Importance Random Forest

Gambar 4.5. menunjukkan nilai *importance* pada sepuluh fitur yang digunakan model *Random Forest*. Sepuluh fitur dengan nilai *importance* tertinggi meliputi beberapa indikator OSLQ seperti GS5, TM1, GS3, TS4, TS3, TS1, TS2, HS1, dan SE3, serta fitur non-psikometrik seperti *Academic Year*. Temuan ini mengindikasikan bahwa prediksi IPK mahasiswa dipengaruhi oleh kombinasi antara faktor perilaku belajar mandiri dan karakteristik akademik mahasiswa, bukan oleh satu jenis fitur saja. Berdasarkan hal tersebut, dapat disimpulkan bahwa model *Random Forest* memiliki kinerja yang baik pada kelas mayoritas, namun kurang sensitif terhadap kelas minoritas, khususnya mahasiswa dengan IPK menengah dan rendah.

4.4.2. Hasil Pengujian Base Model 2: Logistic Regression

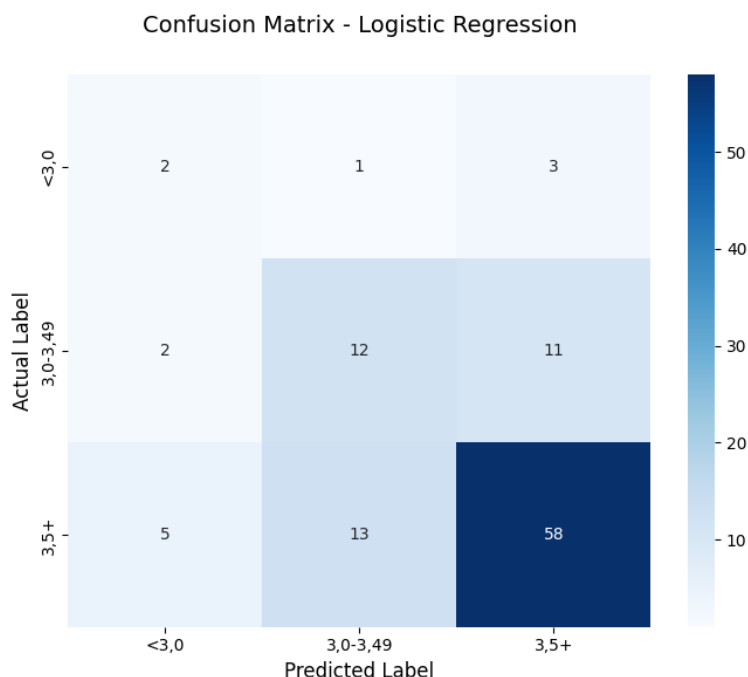
Pengujian kedua dilakukan pada model *Logistic Regression* dengan metode OvR. Model ini mendapat akurasi sebesar 67,29%, lebih rendah dibanding *base model* 1. Berikut adalah tabel *Classification Report* kategori IPK pada *base model* 2:

Tabel 4. 5. *Classification Report Logistic Regression*

Kelas IPK	TP	FP	FN	<i>Precision</i>	<i>Recall</i>	<i>F1-Score</i>	<i>Support</i>
3.5+	58	18	14	0,81	0,76	0,78	76
3.0-3.49	12	13	14	0,46	0,48	0,47	25
<3,0	2	7	4	0,22	0,33	0,27	6
<i>Accuracy</i>						0,67	107
<i>Macro avg</i>				0,29	0,32	0,29	107
<i>Weighted Avg</i>				0,70	0,66	0,68	107

Berdasarkan hasil *classification report* pada tabel 4.5, Kelas IPK 3.5+ merupakan kelas yang paling baik dikenali oleh model *Logistic Regression*, dengan nilai *precision* sebesar 0,81, *recall* sebesar 0,76, dan *F1-Score* sebesar 0,78. Dari total 76 mahasiswa pada kelas ini, sebanyak 58 mahasiswa berhasil diprediksi dengan benar (TP), sementara 14 mahasiswa salah diklasifikasikan ke kelas IPK yang lebih rendah. Hasil ini menunjukkan bahwa *Logistic Regression* cukup andal dalam mengenali pola mahasiswa dengan IPK tinggi, meskipun tingkat kesalahan prediksi masih relatif signifikan. Untuk kelas IPK 3.0–3.49, model *Logistic Regression* menunjukkan performa yang lebih seimbang dibanding *Random Forest*. Kelas ini memiliki *precision* sebesar 0,46, *recall* sebesar 0,48, dan *F1-Score* sebesar 0,47. Dari 25 mahasiswa pada kelas ini, sebanyak 12 mahasiswa berhasil diprediksi dengan benar (TP), sedangkan sisanya mengalami kesalahan klasifikasi ke kelas IPK yang lebih tinggi maupun lebih rendah. Nilai *recall* yang relatif lebih tinggi dibanding *Random Forest* menunjukkan bahwa *Logistic Regression* memiliki sensitivitas yang lebih baik dalam mengenali kelas IPK menengah. Pada kelas IPK <3,0, performa model masih tergolong rendah dengan *F1-Score* sebesar 0,27. Meskipun demikian,

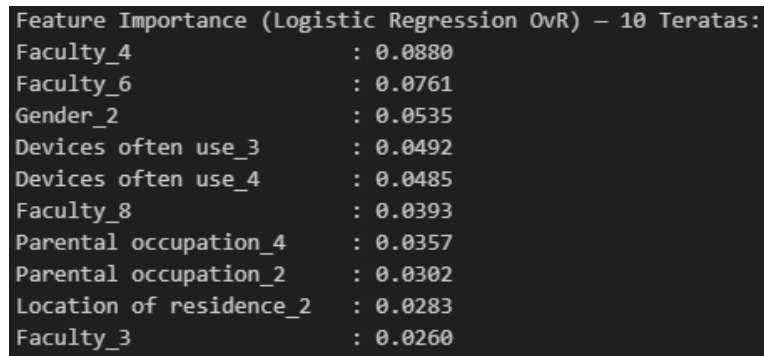
Logistic Regression masih mampu mengidentifikasi 2 mahasiswa secara benar (TP) dari total 6 mahasiswa pada kelas ini. Hasil ini lebih baik dibandingkan *Random Forest* yang sama sekali gagal mengenali kelas IPK rendah. Temuan ini menunjukkan bahwa *Logistic Regression* memiliki kemampuan generalisasi yang lebih baik terhadap kelas minoritas, meskipun performanya belum optimal.



Gambar 4. 6. Confusion Matrix Logistic Regression

Analisis *Confusion Matrix* pada Gambar 4.6 memperkuat temuan tersebut. Dari 76 mahasiswa dengan IPK 3,5+, sebanyak 58 mahasiswa diprediksi dengan benar, sementara 13 mahasiswa salah diklasifikasikan ke kelas 3,0–3,49 dan 5 mahasiswa ke kelas <3,0. Selain itu, terdapat 7 kasus FP pada kelas IPK rendah, yang menunjukkan adanya tumpang tindih batas keputusan antar kelas. Kondisi ini mengindikasikan bahwa model *Logistic Regression* masih kesulitan membedakan secara tegas antara kelas IPK tinggi dan menengah. Perbedaan yang cukup signifikan antara *Macro Average Recall* (0,32) dan *Weighted Average Recall* (0,66) menegaskan bahwa performa model masih didominasi oleh kelas mayoritas. Namun demikian, dibandingkan dengan *Random Forest*, *Logistic*

Regression menunjukkan peningkatan kemampuan dalam mengenali kelas minoritas, khususnya kelas IPK menengah dan rendah.



Feature Importance (Logistic Regression OvR) – 10 Teratas:	
Faculty_4	: 0.0880
Faculty_6	: 0.0761
Gender_2	: 0.0535
Devices often use_3	: 0.0492
Devices often use_4	: 0.0485
Faculty_8	: 0.0393
Parental occupation_4	: 0.0357
Parental occupation_2	: 0.0302
Location of residence_2	: 0.0283
Faculty_3	: 0.0260

Gambar 4. 7. Feature Importance Logistic Regression

Selanjutnya, Gambar 4.7 menampilkan hasil *Feature Importance* pada model *Logistic Regression*. Terlihat bahwa model ini lebih banyak mengandalkan fitur demografis sebagai penentu utama prediksi IPK. Beberapa fitur dengan kontribusi terbesar antara lain *Faculty_4* (Fakultas Teknik), *Faculty_6* (FMIPA), *Gender_2*, serta pola penggunaan perangkat seperti *Devices often use_3* (Smartphone) dan *Devices often use_4* (Tablet). Selain itu, faktor latar belakang keluarga seperti *Parental occupation_4* (*Private*), *Parental occupation_2* (*Self-employed*), dan *Location of residence_2* juga memberikan pengaruh yang cukup signifikan. Secara keseluruhan, hasil ini menunjukkan bahwa model *Logistic Regression* cenderung membangun keputusan prediksi berdasarkan pola linier pada variabel demografis, yang membuatnya lebih sensitif terhadap variasi antar kelas dibandingkan *Random Forest*. Namun, keterbatasan dalam memisahkan kelas secara non-linear menyebabkan performa keseluruhan model masih berada di bawah base model *Random Forest*.

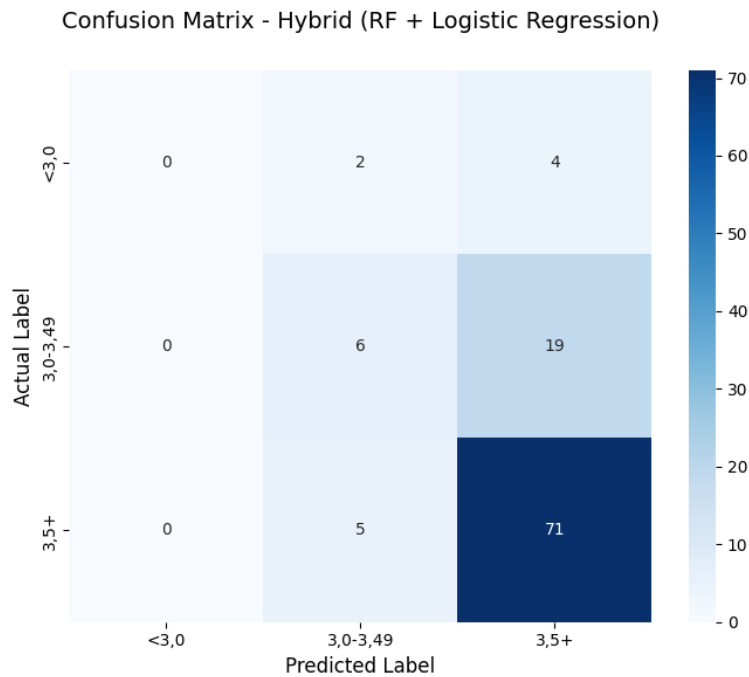
4.4.3. Hasil Pengujian *Hybrid Model*

Pengujian akhir dilakukan pada model usulan *Hybrid Machine Learning*. Dengan menggabungkan prediksi dari kedua *base model*.

Tabel 4. 6. Classification Report Meta Learner

Kelas IPK	TP	FP	FN	<i>Precision</i>	<i>Recall</i>	<i>F1-Score</i>	<i>Support</i>
3.5+	71	5	23	0,76	0,93	0,84	76
3.0-3.49	7	19	7	0,46	0,24	0,32	25
<3,0	0	6	0	0,00	0,00	0,00	6
<i>Accuracy</i>						0,72	107
<i>Macro avg</i>				0,29	0,25	0,25	107
<i>Weighted Avg</i>				0,70	0,75	0,69	107

Berdasarkan hasil evaluasi pada Tabel 4.6, model *Hybrid Machine Learning* yang menggabungkan *Random Forest* dan *Logistic Regression* menghasilkan akurasi sebesar 71,96%, sedikit lebih tinggi dibandingkan *base model Random Forest* (71,03%) dan lebih baik dibandingkan *base model Logistic Regression* (67,29%). Hasil ini menunjukkan bahwa pendekatan *meta learner* mampu memanfaatkan kelebihan masing-masing *base model*, meskipun peningkatan performa yang diperoleh relatif moderat. Nilai *Weighted Average F1-Score* sebesar 0.69 menunjukkan bahwa performa model masih didominasi oleh kelas mayoritas (IPK 3,5+), sementara *Macro Average F1-Score* sebesar 0,25 mengindikasikan bahwa ketidakseimbangan kelas masih menjadi tantangan utama dalam proses klasifikasi. Dengan demikian, meskipun *meta learner* meningkatkan stabilitas prediksi secara keseluruhan, kemampuan generalisasi antar kelas masih terbatas.



Gambar 4. 8. Confusion Matrix Meta Learner

Berdasarkan *confusion matrix* pada Gambar 4.8, model *hybrid* mampu mengklasifikasikan 71 dari 76 mahasiswa dengan IPK 3,5+ secara benar (TP), dengan hanya 5 mahasiswa salah diklasifikasikan sebagai kelas 3,0–3,49 (FN). Hal ini menunjukkan bahwa *meta learner* mempertahankan kemampuan *Random Forest* dalam mengenali kelas IPK tinggi secara konsisten, dengan nilai *recall* mencapai 0,93. Pada kelas IPK 3,0–3,49, *meta learner* berhasil mengidentifikasi 7 mahasiswa secara benar, namun masih terdapat 19 mahasiswa yang salah diklasifikasikan sebagai kelas IPK tinggi. Nilai *recall* sebesar 0,24 menunjukkan bahwa meskipun terdapat perbaikan dibandingkan *Logistic Regression* pada beberapa skenario, model *hybrid* masih cenderung bias terhadap kelas mayoritas. Sementara itu, pada kelas IPK $< 3,0$, model tidak berhasil mengidentifikasi satupun mahasiswa secara benar (*recall* = 0,00). Seluruh data pada kelas ini tetap terklasifikasi ke kelas dengan IPK lebih tinggi. Kondisi ini menegaskan bahwa meskipun *meta learner* mampu meningkatkan akurasi global, performanya

terhadap kelas minoritas masih sangat terbatas akibat distribusi data yang tidak seimbang.

```
Feature Importance (Meta Learner: RF vs LogReg):  
RF_Prob_Class_0      : 0.3027  
RF_Prob_Class_1      : 0.3102  
RF_Prob_Class_2      : 0.3022  
LogReg_Prob_Class_0   : 0.0300  
LogReg_Prob_Class_1   : 0.0096  
LogReg_Prob_Class_2   : 0.0453  
Kontribusi Random Forest : 0.9151  
Kontribusi Logistic Regression : 0.0849
```

Gambar 4. 9. Feature Importance Meta Learner

Berdasarkan hasil analisis *feature importance meta learner* pada Gambar 4.9 menunjukkan bahwa prediksi dari base model *Random Forest* memberikan kontribusi sebesar 91.51%, sedangkan base model *Logistic Regression* hanya berkontribusi sebesar 8.49% terhadap keputusan akhir model *hybrid*. Temuan ini mengindikasikan bahwa *meta learner* jauh lebih mempercayai prediksi *Random Forest* dalam menentukan kelas IPK mahasiswa. Dominasi kontribusi *Random Forest* sejalan dengan performa individual *base model* pada tahap sebelumnya, di mana *Random Forest* menunjukkan kemampuan yang lebih baik dalam menangkap hubungan non-linear dan interaksi antar variabel. Namun demikian, keberadaan *Logistic Regression* tetap memberikan kontribusi pelengkap sebagai model linear yang lebih stabil, khususnya dalam membantu membedakan kelas IPK menengah. Secara keseluruhan, hasil ini menunjukkan bahwa *meta learner* berhasil meningkatkan konsistensi prediksi dibandingkan base model tunggal, namun masih menghadapi keterbatasan dalam mengenali kelas minoritas. Oleh karena itu, strategi lanjutan seperti penyesuaian bobot kelas pada *meta learner* atau pengayaan data kelas minoritas berpotensi meningkatkan sensitivitas model terhadap mahasiswa dengan IPK menengah dan rendah.

4.5. Analisis Hasil

Tahap analisis ini dilakukan untuk melihat temuan dari *feature importance* tiap *base model* untuk mendapatkan wawasan yang bermakna. Analisis dilakukan secara mendalam untuk mengidentifikasi karakteristik dominan, pola tersembunyi, dan faktor-faktor determinan yang membedakan tiap kelas IPK mahasiswa.

Tabel 4.7. Rata-Rata Dimensi Strategi Belajar Daring Berdasarkan Kelas IPK .

IPK	Goal Setting	Environment Structuring	Task Strategies	Time Management	Help Seeking	Self Evaluation
3.5+	3.97	4.22	3.32	3.64	3.65	4.03
3.0- 3.49	3.92	4.10	3.26	3.62	3.63	3.91
2.5- 2.99	3.68	4.13	3.82	3.67	3.82	3.82
2.0- 2.49	3.20	3.38	2.56	3.08	3.88	3.88
<2.0	3.87	4.17	4.00	4.00	4.25	4.17

Berdasarkan hasil analisis pada Tabel 4.7, terlihat bahwa setiap dimensi strategi belajar daring (OSLQ) memiliki variasi nilai rata-rata antar kelas IPK. Mahasiswa dengan IPK di atas 3.5 cenderung menunjukkan nilai yang lebih tinggi pada hampir seluruh dimensi SRL, khususnya pada Goal Setting, Task Strategies, dan Self-Evaluation. Temuan ini mengindikasikan bahwa mahasiswa berprestasi memiliki kemampuan perencanaan belajar, penerapan strategi, serta evaluasi diri yang lebih baik dalam konteks pembelajaran daring. Kesesuaian yang kuat ditemukan ketika hasil deskriptif ini dibandingkan dengan *feature importance* model Random Forest (Gambar 4.5). Sepuluh fitur teratas pada model ini didominasi oleh indikator OSLQ, seperti GS5, GS3, TS1–TS4, TM1, HS1, dan SE3, dengan satu-satunya variabel non-OSLQ yang muncul adalah Academic Year. Dominasi fitur-fitur OSLQ menunjukkan bahwa Random Forest lebih

sensitif terhadap pola perilaku belajar mandiri mahasiswa dibandingkan faktor demografis. Hal ini menunjukkan bahwa model Random Forest mampu menangkap hubungan non-linear dan interaksi kompleks antar dimensi SRL, sehingga perilaku belajar mandiri menjadi faktor utama dalam memprediksi capaian IPK. Keberadaan variabel Academic Year mengindikasikan bahwa tingkat kematangan akademik juga berperan sebagai faktor pendukung, namun kontribusinya tetap berada di bawah pengaruh perilaku belajar. Dengan demikian, hasil ini menegaskan bahwa keberhasilan akademik mahasiswa dalam pembelajaran daring lebih dipengaruhi oleh strategi belajar mandiri daripada karakteristik latar belakang individu, setidaknya dari sudut pandang model Random Forest.

Tabel 4.8. Rata - Rata Faktor Demografi dan Lingkungan Belajar Berdasarkan Kelas IPK

IPK	Faculty	Gender	Parental Education	Parental Occupation	Learning Time During COVID-19	Daily Internet Usage
3.5+	6	1	2	4	1.77	2.69
3.0-3.49	6	1	2	4	1.57	2.59
2.5-2.99	4	1	2	4	1.54	2.73
2.0-2.49	4	2	4	2	1.75	2.25
<2.0	5	1	2	4	1.33	3.00

Berdasarkan tabel 4.8, berbeda dengan Random Forest, hasil feature importance pada model Logistic Regression (OvR) menunjukkan pola yang kontras. Sepuluh fitur teratas pada model ini didominasi oleh variabel demografi dan konteks pembelajaran, seperti Faculty_4, Faculty_6, Gender_2, Devices often use (smartphone dan tablet), Parental Occupation, serta Location of Residence

(Gambar 4.7). Temuan ini menunjukkan bahwa Logistic Regression lebih mengandalkan faktor-faktor struktural yang bersifat linier dalam memisahkan kelas IPK. Latar belakang fakultas mencerminkan perbedaan beban akademik dan karakteristik kurikulum, sementara jenis perangkat yang digunakan berhubungan dengan akses dan kenyamanan dalam mengikuti pembelajaran daring. Selain itu, variabel pekerjaan orang tua dan lokasi tempat tinggal mengindikasikan adanya pengaruh kondisi sosial-ekonomi dan lingkungan belajar terhadap prestasi akademik mahasiswa. Minimnya kemunculan fitur OSLQ pada sepuluh fitur teratas Logistic Regression menunjukkan bahwa model ini kurang sensitif terhadap variasi perilaku belajar yang bersifat kompleks. Hal ini sejalan dengan karakteristik Logistic Regression sebagai model linier, yang cenderung lebih efektif dalam menangkap hubungan langsung dan stabil antar variabel, tetapi kurang mampu merepresentasikan interaksi non-linear antar dimensi perilaku belajar.

Perbedaan pola feature importance antara Random Forest dan Logistic Regression menunjukkan bahwa kedua base model memandang permasalahan prediksi IPK dari perspektif yang berbeda. Random Forest menitikberatkan pada perilaku belajar mandiri (OSLQ) sebagai faktor utama, sementara Logistic Regression lebih menekankan faktor demografi dan konteks pembelajaran. Kondisi ini menjelaskan mengapa pendekatan hybrid melalui meta learner menjadi relevan, karena memungkinkan penggabungan kekuatan kedua model: Random Forest dalam menangkap kompleksitas perilaku belajar dan Logistic Regression dalam merepresentasikan pola struktural yang lebih stabil. Dengan demikian, hasil analisis ini memperkuat bahwa prestasi akademik mahasiswa dalam pembelajaran daring merupakan hasil interaksi antara strategi belajar mandiri dan kondisi demografi serta lingkungan belajar.

4.6. Pembahasan

Hasil penelitian menunjukkan bahwa pendekatan *Hybrid Machine Learning* dengan arsitektur *Stacking Ensemble* yang mengombinasikan *Random*

Forest dan *Logistic Regression* mampu menghasilkan performa yang sedikit lebih baik dibandingkan base model tunggal. Model *hybrid* mencapai akurasi sebesar 71,96%, lebih tinggi dibandingkan *Random Forest* (71,03%) dan *Logistic Regression* (67,29%). Meskipun peningkatan akurasi relatif moderat, temuan ini menunjukkan bahwa integrasi dua model dengan karakteristik berbeda mampu meningkatkan stabilitas prediksi dan mengurangi kelemahan masing-masing model tunggal. Berdasarkan *classification report* dan *confusion matrix*, model *hybrid* menunjukkan performa terbaik pada kelas IPK tinggi (3.5+), dengan nilai recall sebesar 0.93 dan *F1-Score* sebesar 0.84. Hal ini mengindikasikan bahwa model mampu mengenali mahasiswa berprestasi tinggi dengan tingkat kesalahan yang rendah. Namun demikian, performa model pada kelas IPK menengah (3.0–3.49) dan IPK rendah (<3.0) masih terbatas, ditandai dengan nilai recall dan *F1-Score* yang rendah. Kondisi ini menegaskan bahwa ketidakseimbangan distribusi kelas masih menjadi tantangan utama dalam prediksi prestasi akademik mahasiswa. Fenomena tersebut sejalan dengan temuan Irwanto (2024) yang menyatakan bahwa pada dataset pendidikan daring, kelompok mahasiswa berprestasi rendah cenderung *underrepresented*, sehingga algoritma pembelajaran mesin lebih sensitif terhadap kelas mayoritas. Akibatnya, peningkatan akurasi global tidak selalu diikuti dengan peningkatan kemampuan model dalam mengenali kelas minoritas secara konsisten. Analisis *confusion matrix* menunjukkan bahwa sebagian besar kesalahan klasifikasi pada model *hybrid* berasal dari mahasiswa dengan IPK 3.0–3.49 yang diprediksi sebagai kelas 3.5+. Hal ini menunjukkan adanya tumpang tindih batas keputusan antara kelas menengah dan tinggi. Temuan ini mengindikasikan bahwa perbedaan antara kedua kelompok tersebut tidak semata-mata ditentukan oleh capaian akademik, tetapi juga dipengaruhi oleh faktor non-kognitif seperti konsistensi strategi belajar daring, manajemen waktu, dan refleksi diri terhadap proses belajar.

Perbedaan pola *feature importance* antara *Random Forest* dan *Logistic Regression* memberikan wawasan yang lebih mendalam mengenai mekanisme

kerja masing-masing model. Pada *Random Forest*, nilai *feature importance* didominasi oleh indikator *Online Self-Regulated Learning* (OSLQ) seperti *Goal Setting*, *Task Strategies*, *Time Management*, *Help Seeking*, dan *Self-Evaluation*, dengan satu-satunya variabel non-OSLQ yang menonjol adalah *Academic Year*. Hal ini menunjukkan bahwa *Random Forest* lebih efektif dalam menangkap hubungan non-linear dan interaksi kompleks antar perilaku belajar mahasiswa. Temuan ini konsisten dengan teori *Self-Regulated Learning* yang dikemukakan oleh Zimmerman (2002) dan Barnard et al. (2009), yang menekankan bahwa kemampuan menetapkan tujuan, menerapkan strategi belajar, serta melakukan evaluasi diri merupakan determinan utama keberhasilan akademik. Sebaliknya, *Logistic Regression* menunjukkan pola *feature importance* yang didominasi oleh variabel demografi dan konteks pembelajaran, seperti *faculty*, *gender*, *devices often used*, *parental occupation*, dan *location of residence*. Karena bersifat linier dan lebih interpretable, *Logistic Regression* cenderung menangkap hubungan langsung antar variabel tanpa mempertimbangkan interaksi kompleks. Temuan ini mengindikasikan bahwa faktor struktural dan sosial-ekonomi tetap memiliki peran dalam mendukung prestasi akademik, terutama dalam konteks pembelajaran daring yang membutuhkan kesiapan perangkat dan lingkungan belajar yang kondusif. Hal ini sejalan dengan pandangan Pintrich (2000) yang menyatakan bahwa konteks belajar berperan penting dalam membentuk efektivitas strategi regulasi diri.

Pada lapisan *meta learner*, hasil *feature importance* menunjukkan bahwa kontribusi *Random Forest* mencapai 91,51%, sedangkan *Logistic Regression* hanya sebesar 8,49%. Dominasi *Random Forest* mengindikasikan bahwa *meta learner* lebih mempercayai pola non-linear berbasis perilaku belajar yang ditangkap oleh *Random Forest*. Namun demikian, *Logistic Regression* tetap memberikan kontribusi sebagai penyeimbang yang membantu menjaga stabilitas probabilistik dan mengurangi potensi varians model non-linear. Kuhn dan Johnson (2013) menyatakan bahwa model linier dalam kerangka *ensemble* berperan

penting dalam menstabilkan prediksi dan meningkatkan generalisasi. Temuan ini mendukung konsep stacked generalization yang diperkenalkan oleh Wolpert (1992), di mana penggabungan model dengan bias dan varians yang berbeda dapat menghasilkan performa generalisasi yang lebih baik dibandingkan model tunggal. Dalam konteks penelitian ini, pendekatan *hybrid* tidak hanya meningkatkan akurasi prediksi, tetapi juga memperkuat interpretasi pedagogis terhadap faktor-faktor yang memengaruhi prestasi akademik mahasiswa. Secara keseluruhan, pembahasan ini menunjukkan bahwa strategi belajar daring (OSLQ) merupakan determinan utama keberhasilan akademik mahasiswa, sementara faktor demografi dan lingkungan berperan sebagai faktor pendukung. Integrasi pendekatan pembelajaran mesin dengan teori pendidikan memberikan pemahaman yang lebih komprehensif dan bermakna mengenai prestasi akademik mahasiswa. Oleh karena itu, pendekatan hybrid machine learning yang digunakan dalam penelitian ini tidak hanya relevan untuk tujuan prediksi, tetapi juga berpotensi dikembangkan lebih lanjut sebagai dasar sistem learning analytics yang adaptif dan berorientasi pada penguatan strategi belajar mahasiswa.

BAB V

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan hasil penelitian dan pembahasan yang telah dilakukan mengenai prediksi prestasi akademik mahasiswa berbasis strategi belajar daring menggunakan *Hybrid Machine Learning*, maka dapat ditarik kesimpulan sebagai berikut:

1. Strategi belajar daring yang paling berpengaruh terhadap prestasi akademik mahasiswa adalah dimensi-dimensi dalam teori *Self-Regulated Learning* (SRL), khususnya *Goal Setting*, *Task Strategies*, *Help Seeking*, dan *Self-Evaluation*. Keempat indikator tersebut menunjukkan kontribusi tertinggi berdasarkan hasil analisis *feature importance* pada model *Random Forest*. Hal ini membuktikan bahwa mahasiswa yang mampu menetapkan tujuan belajar, mengelola strategi dalam menyelesaikan tugas, meminta bantuan secara aktif, dan melakukan refleksi diri secara teratur memiliki kecenderungan meraih IPK yang lebih tinggi. Temuan ini sejalan dengan teori SRL yang dikemukakan oleh Zimmerman (2002) dan Barnard et al. (2009), bahwa kemampuan regulasi diri dalam belajar merupakan faktor kunci dalam keberhasilan akademik, terutama pada konteks pembelajaran daring.
2. Model *Hybrid Machine Learning* dengan arsitektur *Stacking Ensemble* yang menggabungkan algoritma *Random Forest* dan *Logistic Regression* terbukti efektif untuk memprediksi prestasi akademik mahasiswa berdasarkan strategi belajar daring. Model ini memanfaatkan keunggulan *Random Forest* dalam menangani hubungan non-linear dan keunggulan *Logistic Regression* dalam memberikan interpretasi yang stabil terhadap hasil prediksi. *Meta learner Logistic Regression* mampu mempelajari pola prediksi dari kedua *base model*, menghasilkan keputusan klasifikasi akhir yang lebih konsisten dan adaptif. Analisis *feature importance* pada lapisan *meta learner* menunjukkan bahwa kontribusi model *Random Forest* sebesar 0.9151, sedangkan *Logistic*

Regression sebesar 0,0849, yang berarti model *Random Forest* berperan penting dalam pengambilan keputusan model *hybrid* sedangkan *base model Logistic Regression* hanya berperan sebagai penyeimbang.

3. Performa model *Hybrid* lebih unggul dibandingkan dengan model tunggal. Hasil pengujian menunjukkan bahwa model *Hybrid* mencapai akurasi sebesar 71,96%, lebih tinggi dibandingkan *base model Random Forest* (71%) dan *Logistic Regression* (67,29%). Peningkatan akurasi ini menunjukkan bahwa pendekatan *Stacking Ensemble* berhasil mengurangi kesalahan klasifikasi antar kelas dan meningkatkan kemampuan generalisasi model. Meskipun demikian, model masih mengalami kendala dalam mengenali kelas dengan IPK rendah akibat ketidakseimbangan data. Penggunaan teknik SMOTE membantu memperbaiki distribusi kelas pada data latih, namun jumlah sampel aktual pada kelas minoritas masih terlalu kecil untuk dilatih secara optimal..

5.2. Saran

Berdasarkan hasil penelitian dan keterbatasan yang ditemukan, terdapat beberapa saran yang dapat diajukan untuk pengembangan penelitian selanjutnya:

1. Penelitian selanjutnya disarankan untuk menambah jumlah sampel data, khususnya pada kategori mahasiswa dengan IPK rendah (<2.5), agar model dapat mempelajari pola perilaku belajar mahasiswa yang berisiko secara lebih mendalam tanpa terlalu bergantung pada data sintesis yang dihasilkan metode penanganan ketidakseimbangan data.
2. Peneliti selanjutnya dapat menerapkan metode pencarian parameter otomatis seperti *Grid Search* atau *Randomized Search* untuk menemukan kombinasi parameter yang lebih optimal pada lapisan *base model* maupun *Meta Learner*.
3. Peneliti selanjutnya dapat menambahkan variabel prediktor lain yang tidak terbatas pada kuesioner mandiri, seperti data aktivitas *log* dari LMS.
4. Peneliti selanjutnya disarankan untuk mengeksplorasi algoritma *Boosting* sebagai *base model* dalam skema *stacking* untuk melihat apakah ada peningkatan yang lebih signifikan pada model tersebut.

DAFTAR PUSTAKA

- Alias, M. A. H., Hambali, N., Abdul Aziz, M. A., Taib, M, N., & Jailani, R. (2024). Feature Selection Techniques and Classification Algorithms for Student Performance Classification: A Review. *International Journal of Electrical and Computer Engineering (IJECE)*, 14(3), 3230-3243.
- Ardiningtyas, F., & Rosa, S. F. (2021). Analisis Perbandingan Algoritma Klasifikasi Machine Learning pada Data Tidak Seimbang (Studi Kasus: Prediksi Churn Pelanggan Telekomunikasi).
- Barnard, L., Lan, W. Y., To, Y. M., Paton, V. O., & Lai, S. L. (2009). Measuring self-regulation in online learning environments. *Distance Education*, 30(1), 5-14.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*.
- Breiman, L. (2001). Random Forests. [Machine Learning, 45(1), 5-32.
- Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth
- Freidman, J. (1984). A Recursive Partitioning Algorithm for Prediction and Classification. *Journal of the American Statistical Association*, 79(387), 548-558.
- Freidman, J. (2009). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189-1232.
- Gareth, J., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning with Applications in R*.
- Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.).
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning: with Applications in R*

- Khemchandani, R., Kumar, A., & Gupta, S. (2021). Two-Stage Machine Learning Framework for Predicting Student Performance. *Procedia Computer Science*, 184, 827-833.
- Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*.
- Kutner, M. H., Nachtsheim, C. J., Neter, J., & Li, W. (2015). *Applied Linear Statistical Models* (5th ed.). McGraw-Hill Education.
- Mohri, M., Rostamizadeh, A., & Talwalkar, A. (2018). *Foundations of Machine Learning* (2nd ed.). The MIT Press
- Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. The MIT Press
- Nasution, R. K., Sibarani, T. D., & Purba, P. R. (2019). Perbandingan Metode Normalisasi Data Pada Klasifikasi Menggunakan Algoritma K-Nearest Neighbor. *Jurnal Ilmu Komputer dan Informasi*, 12(2), 79-86.
- Pintrich, P. R. (2000). The role of goal orientation in self-regulated learning. In M. Boekaerts, P. R. Pintrich, & M. Zeidner (Eds.), *Handbook of Self-Regulation* (pp. 451-502).
- Powers, D. M. W. (2011). Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness & Correlation. *Journal of Machine Learning Technologies*, 2(1), 37-63.
- Quinlan, J. R. (1986). Induction of Decision Trees. *Machine Learning*, 1(1), 81-106.
- Refaeilzadeh, M., Tang, L., & Liu, H. (2009). Cross-Validation. In L. Liu & M. T. Özsu (Eds.), *Encyclopedia of Database Systems* (pp. 532-538).
- Rohman, A., Sari, Y. A., & Wibowo, A. T. (2025). Prediksi Prestasi Akademik Mahasiswa Berdasarkan Strategi Belajar Online Menggunakan Hybrid Machine Learning.
- Rohman, M.G., Abdullah, Z., & Kasim, S. (2025). Hybrid *Logistic Regression* Random Forest on Predicting Student Performance. *JOIV : International Journal on Informatics Visualization*, 9(2), 852-858
- Rokach, L. (2010). Ensemble-based classifiers. In M. Maimon & L. Rokach (Eds.), *Data Mining and Knowledge Discovery Handbook* (pp. 777-802).

- Saadah, S., & Salsabila, N. (2021). Perbandingan Algoritma Decision Tree, Random Forest, dan Naïve Bayes untuk Klasifikasi Data Titanic. *Jurnal Ilmiah Teknik Informatika*, 5(2), 1-8.
- Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3), 210-229.
- Santra, R., & Christy, K. (2012). Random Forest Classifier for Medical Diagnosis. *International Journal of Computer Applications*, 46(20), 40-44.
- Sari, Y. A. (2018). Penanganan Data Missing Value pada Dataset Diabetes Menggunakan Metode Imputasi Mean, Median, dan Mode. *Jurnal Teknologi Informasi dan Ilmu Komputer*, 5(2), 127-134.
- Siringoringo, F. P. (2018). Implementasi Metode SMOTE (Synthetic Minority Oversampling Technique) untuk Mengatasi Imbalance Class pada Klasifikasi Penyakit Jantung. *Jurnal Rekayasa Sistem Industri*, 7(1), 21-28.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nded.).
- Tanjung, A. A. S., & Kartiko, A. (2017). Pohon Klasifikasi dan Regresi (Classification and Regression Tree) pada Data Produksi Padi Jawa Timur Tahun 2015. *Jurnal Gaussian*, 6(4), 517-526.
- Tharwat, A. (2021). Classification Performance Metrics. *Applied Computing and Informatics*, 17(1), 108-121.
- Tibishirani, R., & Freidman, J. (2019). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*.
- Vegari, M., & Budi, A. S. (2020). Penerapan Exploratory Data Analysis (EDA) dalam Analisis Sentimen Ulasan Film Menggunakan Naïve Bayes Classifier. *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, 4(2), 241-247.
- Wahyuni, S., Rahmat, R. F., & Siregar, A. M. (2019). Perancangan Sistem Informasi E-Learning Berbasis Web Menggunakan Metode Prototype. *Jurnal Teknologi Informasi & Pendidikan*, 12(1), 47-56.
- Wolpert, D. H. (1992). Stacked Generalization. *Neural Networks*, 5(2), 241-259.

- Zhang, G. P. (2011). Neural networks for time series forecasting. *International Journal of Forecasting*, 27(2), 335-364.
- Zimmerman, B. J. (2002). Becoming a Self-Regulated Learner: An Overview. *Theory Into Practice*, 41(2), 64-70.

LAMPIRAN

Lampiran 1. Github

https://github.com/coretail/Skripsi_Abdullah_Dzaki

Lampiran 2. *Source Code*

```
ordinal_cols = [  
    'Age group', 'Academic year', 'Cumulative average (GPA)',  
    'Daily Internet usage', 'Learning time before COVID-19',  
    'Learning time during COVID-19', 'Parental education', 'No. of  
Siblings'  
]  
  
nominal_cols = [  
    'Faculty', 'Gender', 'Location of residence',  
    'Source of tuition fees ', 'Devices often use', 'Parental  
occupation'  
]  
  
# encode nominal  
df_encoded = pd.get_dummies(df, columns=nominal_cols,  
drop_first=True)  
  
# encode ordinal  
ord_enc = OrdinalEncoder()  
df_encoded[ordinal_cols] =  
ord_enc.fit_transform(df_encoded[ordinal_cols])  
  
def merge_ipk_raw(label):  
    if label == 0:  
        return 'tinggi'  
    elif label in [1]:  
        return 'menengah'  
    elif label in [2, 3, 4]:  
        return 'rendah'  
    else:  
        return None  
  
df_encoded['IPK_group'] = df_encoded['Cumulative average  
(GPA)'].apply(merge_ipk_raw)  
print(df_encoded['IPK_group'].value_counts(dropna=False))  
ipk_final_map = {  
    'rendah': 0,  
    'menengah': 1,  
    'tinggi': 2
```



```

}

df_encoded['IPK_final'] = df_encoded['IPK_group'].map(ipk_final_map)

print("=== DISTRIBUSI IPK FINAL ===")
print(df_encoded['IPK_final'].value_counts().sort_index())

print("\n=== PERSENTASE ===")
print(
    df_encoded['IPK_final']
    .value_counts(normalize=True)
    .sort_index() * 100
)
oslq_cols = [col for col in df_encoded.columns if
col.startswith(('GS', 'EH', 'TS', 'TM', 'HR', 'SE'))]
print("Kolom OSLQ:", oslq_cols)

# scaling
scaler = MinMaxScaler()

# Fit dan transform kolom OSLQ
df_encoded[oslq_cols] = scaler.fit_transform(df_encoded[oslq_cols])

# Membagi data menjadi train (80%) dan test (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

label_mapping = {
    0: "<3.0",
    1: "3.0-3.49",
    2: "3.5+"
}

# Inisialisasi SMOTE
smote = SMOTE(random_state=42, k_neighbors=1)

# Terapkan SMOTE hanya pada data training
X_train_resampled, y_train_resampled = smote.fit_resample(X_train,
y_train)

# fungsi gini index
def gini_index(groups, classes):
    n_instances = float(sum([len(group) for group in groups]))
    gini = 0.0
    for group in groups:
        size = float(len(group))
        if size == 0:
            continue
        score = 0.0
        class_values = [row[-1] for row in group]

```

```

        for class_val in classes:
            p = class_values.count(class_val) / size
            score += p * p
            gini += (1.0 - score) * (size / n_instances)
    return gini
def test_split(index, value, dataset):
    left, right = [], []
    for row in dataset:
        if row[index] < value:
            left.append(row)
        else:
            right.append(row)
    return left, right
# fungsi split data
def get_split(dataset):
    class_values = list(set(row[-1] for row in dataset))
    b_index, b_value, b_score, b_groups = 999, 999, 999, None
    for index in range(len(dataset[0]) - 1):
        for row in dataset:
            groups = test_split(index, row[index], dataset)
            gini = gini_index(groups, class_values)
            if gini < b_score:
                b_index, b_value, b_score, b_groups = index,
row[index], gini, groups
    return {'index': b_index, 'value': b_value, 'groups': b_groups}
# fungsi node terminal
def to_terminal(group):
    outcomes = [row[-1] for row in group]
    return max(set(outcomes), key=outcomes.count)
# fungsi split node rekursif
def split(node, max_depth, min_size, depth):
    left, right = node['groups']
    del (node['groups'])

    # jika salah satu sisi kosong -> buat leaf
    if not left or not right:
        outcomes = [row[-1] for row in left + right]
        node['left'] = node['right'] = max(set(outcomes),
key=outcomes.count)
        return

    # stop jika kedalaman maksimum
    if depth >= max_depth:
        node['left'], node['right'] = to_terminal(left),
to_terminal(right)
        return

    # split sisi kiri dan kanan
    if len(left) <= min_size:
        node['left'] = to_terminal(left)

```

```

else:
    node['left'] = get_split_rf(left, n_features)
    # ↓ tambahkan ini supaya gain tersimpan di semua node bawah
    split(node['left'], max_depth, min_size, depth + 1)

if len(right) <= min_size:
    node['right'] = to_terminal(right)
else:
    node['right'] = get_split_rf(right, n_features)
    # ↓ sama, recursive untuk kanan
    split(node['right'], max_depth, min_size, depth + 1)
# fungsi build tree
def build_tree(train, max_depth, min_size):
    root = get_split(train)
    split(root, max_depth, min_size, 1)
    return root
# fungsi predict
def predict(node, row):
    if row[node['index']] < node['value']:
        if isinstance(node['left'], dict):
            return predict(node['left'], row)
        else:
            return node['left']
    else:
        if isinstance(node['right'], dict):
            return predict(node['right'], row)
        else:
            return node['right']
# pipeline & testing
def decision_tree(train, test, max_depth, min_size):
    tree = build_tree(train, max_depth, min_size)
    predictions = [predict(tree, row) for row in test]
    return predictions
# fungsi bootstrap sample
def subsample(dataset, ratio):
    sample_size = round(len(dataset) * ratio)
    sample_data = []
    while len(sample_data) < sample_size:
        index = randrange(len(dataset))
        sample_data.append(dataset[index])
    return sample_data
# get_split_rf dengan Gini Gain
def get_split_rf(dataset, n_features):
    class_values = list(set(row[-1] for row in dataset))
    features = sample(range(len(dataset[0]) - 1), n_features)
    b_index, b_value, b_score, b_groups, best_gain = 999, 999, 999,
    None, 0

    current_gini = gini_index([dataset], class_values)

```

```

    for index in features:
        for row in dataset:
            groups = test_split(index, row[index], dataset)
            gini_after = gini_index(groups, class_values)
            gain = current_gini - gini_after # impurity reduction

            if gini_after < b_score:
                b_index, b_value, b_score, b_groups, best_gain =
index, row[index], gini_after, groups, gain

    return {
        'index': b_index,
        'value': b_value,
        'groups': b_groups,
        'gain': best_gain
    }

# build tree dengan split_rf
def build_tree_rf(train, max_depth, min_size, n_features):
    root = get_split_rf(train, n_features)
    split(root, max_depth, min_size, 1)
    return root

# Pindahkan fungsi build_one_tree global
def build_one_tree(train, max_depth, min_size, sample_size,
n_features):
    sample = subsample(train, sample_size)
    tree = build_tree_rf(sample, max_depth, min_size, n_features)
    return tree

def build_forest_parallel(train, max_depth, min_size, sample_size,
n_trees, n_features, n_jobs=-1):
    print(f" Membangun {n_trees} pohon secara paralel ({n_jobs} if
n_jobs != -1 else 'semua CPU') core aktif")

    # Gunakan tqdm untuk progress bar per pohon
    results = Parallel(n_jobs=n_jobs)(
        delayed(build_tree_rf)(subsample(train, sample_size),
max_depth, min_size, n_features)
        for _ in tqdm(range(n_trees), desc="Building Trees",
ncols=80)
    )

    print(f" Semua {n_trees} pohon selesai dibangun!")
    return results

# voting
def bagging_predict(trees, row):
    predictions = [predict(tree, row) for tree in trees]
    return max(set(predictions), key=predictions.count)

def bagging_predict_proba(forest, row, n_classes):
    """
    Menghitung probabilitas kelas dari Random Forest custom
    """

```

```

predictions = [predict(tree, row) for tree in forest]
counts = Counter(predictions)

proba = np.zeros(n_classes)
for cls, cnt in counts.items():
    proba[int(cls)] = cnt / len(forest)

    return proba
# pipeline random forest
def random_forest(train, test, max_depth, min_size, sample_size,
n_trees, n_features, n_jobs=None):
    forest = build_forest_parallel(train, max_depth, min_size,
sample_size, n_trees, n_features, n_jobs)
    predictions = [bagging_predict(forest, row) for row in test]
    return predictions
# Gabungkan fitur dan target jadi satu dataset
train_data = [list(X_train_resampled.iloc[i]) +
[y_train_resampled.iloc[i]] for i in range(len(X_train_resampled))]
test_data = [list(X_test.iloc[i]) + [y_test.iloc[i]] for i in
range(len(X_test))]
# logistic regression (base model)
# fungsi Sigmoid
def sigmoid(z):
    return 1 / (1 + math.exp(-z))
def predict_logreg(x, weights):
    z = weights[0]
    for i in range(len(x)):
        z += weights[i + 1] * x[i]
    return 1.0 / (1.0 + np.exp(-z))
def train_logistic_regression(train, l_rate, n_epoch, patience=100,
min_delta=0.01):
    weights = [0.0 for _ in range(len(train[0]))]
    error_history = []
    best_error = float('inf')
    no_improve = 0

    for epoch in range(n_epoch):
        sum_error = 0
        for row in train:
            x = np.array(row[:-1], dtype=float) # fitur numerik
            y = float(row[-1]) # label

            yhat = predict_logreg(x, weights)
            error = y - yhat
            sum_error += float(np.sum(error ** 2))

            weights[0] += l_rate * error * yhat * (1.0 - yhat)
            for i in range(len(x)):
                weights[i + 1] += l_rate * error * yhat * (1.0 -
yhat) * x[i]

```

```

error_history.append(sum_error)

if epoch % 100 == 0:
    print(f"Epoch={epoch}, Error={sum_error:.3f}")

if sum_error < best_error - min_delta:
    best_error = sum_error
    no_improve = 0
else:
    no_improve += 1

if no_improve >= patience:
    print(f" Early stopping di epoch {epoch} (error tidak
membaik selama {patience} epoch).")
    break

return weights, error_history
# Prediksi Logistic Regression
def logistic_regression(train, test, l_rate, n_epoch, patience=100,
min_delta=0.01, return_weights=False):
    weights, error_history = train_logistic_regression(
        train, l_rate, n_epoch,
        patience=patience, min_delta=min_delta
    )

    predictions = [round(predict_logreg(row, weights)) for row in
test]

    if return_weights:
        return predictions, error_history, weights
    else:
        return predictions, error_history
def predict_multiclass(test, models):
    predictions = []
    for row in test:
        x = np.array(row[:-1], dtype=float) # fitur saja
        probs = [predict_logreg(x, weights) for weights in models]
        predictions.append(int(np.argmax(probs)))
    return predictions

def predict_proba_multiclass(data, models):
    """
    Menghasilkan probabilitas kelas untuk Logistic Regression OvR
(custom)
    Struktur model: [w1, w2, ..., wN, bias]
    """
    probas = []

    for row in data:

```

```

# fitur saja (tanpa label)
x = np.array(row[:-1], dtype=float)
scores = []

for model in models:
    weights = np.array(model[:-1], dtype=float)
    bias = model[-1]
    z = bias + np.dot(weights, x)
    scores.append(z)

# softmax (numerically stable)
scores = np.array(scores)
exp_scores = np.exp(scores - np.max(scores))
probs = exp_scores / np.sum(exp_scores)

probas.append(probs)

return np.array(probas)
# Fungsi OvR Multi-class Logistic Regression (paralel + early
# stopping + error tracking)
def logistic_regression_multiclass(train, test, n_classes, l_rate,
n_epoch, n_jobs=4, patience=100, min_delta=0.01):
    """
    Multi-class Logistic Regression (One-vs-Rest) dengan:
    - Early stopping berdasarkan error
    - Paralel training menggunakan joblib
    - Tracking error tiap kelas
    """
    error_histories = [] # simpan semua history error untuk tiap
    kelas

    def train_one_vs_rest(class_val):
        # ubah label ke binary: 1 kalau row[-1] == class_val, 0
        kalau bukan
        train_binary = []
        for row in train:
            new_row = list(row)
            new_row[-1] = 1.0 if row[-1] == class_val else 0.0
            train_binary.append(new_row)

        # latih logistic regression dengan early stopping
        weights, err_hist = train_logistic_regression(
            train_binary, l_rate, n_epoch,
            patience=patience, min_delta=min_delta
        )
        return weights, err_hist

    print(f" Melatih {n_classes} model Logistic Regression secara
    paralel (dengan early stopping)...")

```

```

# training paralel dengan joblib
results = Parallel(n_jobs=n_jobs) (
    delayed(train_one_vs_rest)(class_val)
    for class_val in tqdm(range(n_classes), desc="Training OvR
Models", ncols=80)
)

# pisahkan hasil training (weights dan error histories)
models = [r[0] for r in results]
error_histories = [r[1] for r in results]

print(" Semua model OvR selesai dilatih!\n")

return models, error_histories
# Parameter Random Forest
n_trees = 128
max_depth = 10
min_size = 5
sample_size = 1.0
n_features = int(sqrt(len(X_train_resampled.columns)))
# Base Model 1: Random Forest (Paralel)
forest = build_forest_parallel(
    train_data,
    max_depth,
    min_size,
    sample_size,
    n_trees,
    n_features,
    n_jobs=4
)

n_classes = 3

rf_proba_train = np.array([
    bagging_predict_proba(forest, row, n_classes)
    for row in train_data
])

rf_proba_test = np.array([
    bagging_predict_proba(forest, row, n_classes)
    for row in test_data
])

# Base Model 2: Logistic Regression (One-vs-Rest Multi-class)
# Gabungkan data training & testing ke format list
train_logreg = [list(X_train_resampled.iloc[i]) +
[y_train_resampled.iloc[i]] for i in range(len(X_train_resampled))]
test_logreg = [list(X_test.iloc[i]) + [y_test.iloc[i]] for i in
range(len(X_test))]

# Jumlah kelas (IPK 0-4)

```



```

n_classes = 3

# Latih Logistic Regression OvR (sekali saja)
models, train_error_histories = logistic_regression_multiclass(
    train_logreg,
    train_logreg,
    n_classes,
    l_rate=0.0001,
    n_epoch=2000,
    patience=100,
    min_delta=0.01,
    n_jobs=4
)

# Prediksi train dan test dengan model hasil training
y_pred_logreg_train = predict_multiclass(train_logreg, models)
y_pred_logreg_test = predict_multiclass(test_logreg, models)

logreg_proba_train = predict_proba_multiclass(train_logreg, models)
logreg_proba_test = predict_proba_multiclass(test_logreg, models)

# Cek error histories tiap kelas
for i, history in enumerate(train_error_histories):
    print(f" Class {i} | Epochs = {len(history)} | Final Error = {history[-1]:.4f}")
acc_test = (np.array(y_pred_logreg_test) == y_test.values).mean() * 100
print(f"Akurasi Logistic Regression (Test): {acc_test:.2f}%")
# Buat Meta Dataset (gabungan hasil prediksi dua base model)
meta_train = []
for i in range(len(y_train_resampled)):
    row = (
        list(rf_proba_train[i]) +
        list(logreg_proba_train[i]) +
        [int(y_train_resampled.values[i])]
    )
    meta_train.append(row)

meta_test = []
for i in range(len(y_test)):
    row = (
        list(rf_proba_test[i]) +
        list(logreg_proba_test[i]) +
        [int(y_test.values[i])]
    )
    meta_test.append(row)

print(len(meta_train[0]))
print(meta_train[0])

```

```

print("Last element:", meta_train[0][-1], type(meta_train[0][-1]))
y_meta_train = y_train_resampled.values

# Jumlah kelas target IPK (1-3)
n_classes = 3

# Meta Learner: Logistic Regression (Multi-class OvR) dengan Early Stopping
meta_models, meta_error_histories = logistic_regression_multiclass(
    train=meta_train,
    test=None,
    n_classes=n_classes,
    l_rate=0.001,
    n_epoch=250,
    patience=100,
    min_delta=0.01,
    n_jobs=4
)

# Prediksi hasil meta learner (pakai model yang sama)
y_pred_final = predict_multiclass(meta_test, meta_models)

# Cek isi error histories (jumlah epoch yang dijalankan tiap kelas)
for i, history in enumerate(meta_error_histories):
    print(f" Class {i} | Epochs = {len(history)} | Final Error = {history[-1]:.4f}")

# Evaluasi manual akurasi model hybrid
accuracy = (y_pred_final == y_test.values).mean() * 100

print(f"\n Akurasi Model Hybrid (RF + LogReg OvR): {accuracy:.2f}%")
meta_cols = [
    "RF_p0", "RF_p1", "RF_p2",
    "LR_p0", "LR_p1", "LR_p2", "actual"
]

meta_train_df = pd.DataFrame(meta_train, columns=meta_cols)
meta_test_df = pd.DataFrame(meta_test, columns=meta_cols)

print(meta_train_df.head())
print(meta_test_df.head())
ipk_labels = ["<3,0", "3,0-3,49", "3,5+"]

def evaluate_model(y_true, y_pred, model_name="Model",
labels_custom=None, normalize=False):
    print(f"\n=== {model_name} Evaluation ===")
    print(f"Accuracy: {accuracy_score(y_true, y_pred)*100:.2f}%\n")

    classes = [0, 1, 2]

```

```

# === Classification Report ===
if labels_custom and len(labels_custom) == len(classes):
    print("Classification Report:")
    print(classification_report(
        y_true, y_pred,
        labels=classes,
        target_names=labels_custom,
        digits=2
    ))
    tick_labels = labels_custom
else:
    print("Classification Report:")
    print(classification_report(y_true, y_pred, labels=classes,
digits=2))
    tick_labels = classes

# === Confusion Matrix ===
cm = confusion_matrix(
    y_true, y_pred,
    labels=classes,
    normalize='true' if normalize else None
)

plt.figure(figsize=(7, 6))
sns.heatmap(
    cm, annot=True,
    fmt=".2f" if normalize else "d",
    cmap="Blues",
    xticklabels=tick_labels,
    yticklabels=tick_labels
)
plt.title(f"Confusion Matrix - {model_name}\n{'(Normalized)' if
normalize else ''}", fontsize=14, pad=10)
plt.xlabel("Predicted Label", fontsize=12)
plt.ylabel("Actual Label", fontsize=12)
plt.tight_layout()
plt.show()
print([v for v in globals() if "rf" in v.lower()])
# evaluasi random forest
y_pred_rf_train = np.argmax(rf_proba_train, axis=1)
y_pred_rf_test  = np.argmax(rf_proba_test, axis=1)

evaluate_model(
    y_true=y_test.values,
    y_pred=y_pred_rf_test,
    model_name="Random Forest",
    labels_custom=ipk_labels,
    normalize=False
)

```

```

# evaluasi logistic regression
actual_lr = [row[-1] for row in test_logreg]
evaluate_model(actual_lr, y_pred_logreg_test, model_name="Logistic
Regression", labels_custom=ipk_labels)
print(type(y_test.values[0]), y_test.values[:5])
print(type(y_pred_final[0]), y_pred_final[:5])
evaluate_model(
    y_true=y_test.values,
    y_pred=y_pred_final,
    model_name="Hybrid (RF + Logistic Regression)",
    labels_custom=ipk_labels,
    normalize=False
)
# Random forest feature importance (base model 1)

def compute_rf_feature_importance(forest, n_total_features):
    importance = np.zeros(n_total_features)

    def traverse(node):
        if isinstance(node, dict):
            idx = node['index']
            gain = node.get('gain', 0)
            importance[idx] += gain
            traverse(node['left'])
            traverse(node['right'])

    for tree in forest:
        traverse(tree)

    total = np.sum(importance)
    if total > 0:
        importance /= total
    return importance
# feature importance (base model 2)

def compute_logreg_feature_importance(models, feature_names=None):
    """Hitung importance rata-rata dari semua model OvR Logistic
    Regression"""
    # Ambil bobot tiap model OvR (models berisi list of weights)
    weights = np.array([np.abs(model[1:]) for model in models]) #
    skip bias di index 0
    mean_importance = np.mean(weights, axis=0)

    # Normalisasi
    mean_importance /= np.sum(mean_importance)

    print("\nFeature Importance (Logistic Regression OvR):")
    if feature_names is not None:
        for name, imp in zip(feature_names, mean_importance):
            print(f"{name:<25} → {imp:.4f}")

```

```

    else:
        for i, imp in enumerate(mean_importance):
            print(f"Feature {i+1}: {imp:.4f}")

    return mean_importance
# ambil feature importance
rf_importance = compute_rf_feature_importance(forest,
len(X_train_resampled.columns))
# Buat array nama fitur & importance
feature_names = np.array(X_train_resampled.columns)
importances = np.array(rf_importance)

# Urutkan dari terbesar ke terkecil
sorted_idx = np.argsort(importances)[::-1]
sorted_features = feature_names[sorted_idx]
sorted_importances = importances[sorted_idx]

# Cetak hasil terurut
print("\n Feature Importance (Random Forest) - Diurutkan:")
for f, imp in zip(sorted_features, sorted_importances):
    print(f"{f:<30} → {imp:.4f}")
# Ambil hasil importance
logreg_importance = compute_logreg_feature_importance(models,
X_train_resampled.columns)
# Urutkan dari yang terbesar ke terkecil
sorted_features = sorted(
    zip(X_train_resampled.columns, logreg_importance),
    key=lambda x: x[1],
    reverse=True
)

print("\n Feature Importance (Logistic Regression OvR) -
Diurutkan:")
for name, imp in sorted_features:
    print(f"{name:<25} → {imp:.4f}")
def compute_meta_feature_importance(meta_models,
feature_names=None):
    """
    Menghitung rata-rata absolut bobot dari model meta-learner (OvR
    Logistic Regression)
    untuk melihat kontribusi masing-masing base model.
    """
    # Ambil bobot tiap model (kecuali bias di indeks 0)
    weights = np.array([np.abs(model[1:]) for model in meta_models])
# shape: [n_classes, n_features]

# Hitung rata-rata per fitur di seluruh kelas
mean_importance = np.mean(weights, axis=0)

# Normalisasi agar total = 1

```

```

mean_importance /= np.sum(mean_importance)

# Tampilkan hasil
print("Feature Importance (Meta Learner: RF vs LogReg):")
if feature_names is not None:
    for name, imp in zip(feature_names, mean_importance):
        print(f"{name:<25} : {imp:.4f}")
else:
    for i, imp in enumerate(mean_importance):
        print(f"Feature {i+1:<2} : {imp:.4f}")

    return mean_importance
# Buat array nama fitur & importance
feature_names = np.array(X_train_resampled.columns)
importances = np.array(rf_importance)

# Urutkan dari yang terbesar ke terkecil
sorted_idx = np.argsort(importances)[::-1]
sorted_features = feature_names[sorted_idx]
sorted_importances = importances[sorted_idx]

# Ambil hanya 10 teratas
top_n = 10
sorted_features = sorted_features[:top_n]
sorted_importances = sorted_importances[:top_n]

# Cetak hasil
print("Feature Importance (Random Forest) - 10 Teratas:")
for f, imp in zip(sorted_features, sorted_importances):
    print(f"{f:<30} : {imp:.4f}")
# Urutkan dari yang terbesar ke terkecil
sorted_features = sorted(
    zip(X_train_resampled.columns, logreg_importance),
    key=lambda x: x[1],
    reverse=True
)[:10] # ambil 10 teratas

print("Feature Importance (Logistic Regression OvR) - 10 Teratas:")
for name, imp in sorted_features:
    print(f"{name:<25} : {imp:.4f}")
# Nama meta-feature (3 RF + 3 LogReg)
meta_feature_names = [
    "RF_Prob_Class_0",
    "RF_Prob_Class_1",
    "RF_Prob_Class_2",
    "LogReg_Prob_Class_0",
    "LogReg_Prob_Class_1",
    "LogReg_Prob_Class_2"
]

```

```

# Hitung feature importance meta learner
meta_importance = compute_meta_feature_importance(
    meta_models,
    feature_names=meta_feature_names
)

rf_contribution = meta_importance[:3].sum()
lr_contribution = meta_importance[3:].sum()

print(f"Kontribusi Random Forest      : {rf_contribution:.4f}")
print(f"Kontribusi Logistic Regression : {lr_contribution:.4f}")

```